

CMS для парикмахерских и барбершопов «Сайтама»

РУКОВОДСТВО ПО СТРУКТУРЕ ПРОЕКТА

Подробный разбор кодовой базы для тех, кто только
начинает в ней разбираться

Версия документа 1.0

Каждый файл backend/ и frontend/ – что делает и как читать его
код

С примерами кода и пояснениями по архитектуре

Составлено по состоянию кода на август 2026

О ЧЁМ ЭТОТ ДОКУМЕНТ

У проекта CMS «Сайтама» уже есть четыре документа: техническое задание (что нужно было сделать), руководство пользователя и руководство администратора (как пользоваться готовой системой) и итоговый отчёт (что в итоге сделано и почему именно так). Все они описывают *продукт* – то, что видит клиент, администратор или проверяющий.

Этот документ устроен иначе. Он описывает *код* – репозиторий `fastapi_hair_salon` файл за файлом: какая папка за что отвечает, какой файл что делает, как одна функция вызывает другую от HTTP-запроса браузера до строки в базе данных, и почему многие вещи в коде сделаны именно так, а не иначе. Адресован новому разработчику (или самому себе через полгода), который клонировал репозиторий и не знает, с чего начать чтение.

Что нужно знать заранее. Базовые понятия программирования: переменные, функции, классы, HTTP-запросы, что такое база данных и таблица. Знание Python, FastAPI, Vue или SQL *не требуется* – каждое новое понятие (слоистая архитектура, ORM, Pydantic-схема, Vue-компонент, Pinia-хранилище и т.д.) объясняется в тексте при первом появлении, на примере реального кода этого проекта.

Как читать этот документ.

– Разделы 1–3 – один раз подряд, от начала до конца: они дают общую картину (что за проект, из каких частей состоит, как эти части вызывают друг друга на сквозном примере одной записи на стрижку). Без этой картины отдельные файлы не складываются в целое.

– Разделы 4–21 – как справочник: открыли файл в редакторе, нашли его путь в оглавлении, прочитали описание. Каждый файл `backend/` и `frontend/` описан отдельным блоком: назначение, ключевые функции/классы с сигнатурами и (если есть что-то неочевидное) отдельная пометка «Особенности».

– Раздел 21 в конце – куда идти дальше, если этого документа не хватило (README, ROADMAP, остальные документы проекта).

Чего в этом документе нет. Пошаговых инструкций «как пользователю записаться на стрижку» (это – руководство пользователя) и инструкций по развёртыванию на боевом сервере (это – руководство администратора, раз-

дел 19 этого документа даёт только краткую версию для локального запуска).
Здесь – только код и то, как он устроен.

Содержание

О ЧЁМ ЭТОТ ДОКУМЕНТ	1
1 Обзор проекта	6
Что делает система	6
Роли пользователей	6
Технологический стек	7
Слоистая архитектура backend	8
Frontend: Vue 3 без TypeScript	10
2 Структура репозитория верхнего уровня	11
3 Backend: архитектура и жизненный цикл приложения	14
3.1 Точка входа: <code>main.py</code> и <code>config.py</code>	14
3.2 Сессия БД и фоновые задачи: <code>database.py</code> и <code>scheduler.py</code>	16
3.3 Слои на примере одной записи: от HTTP-запроса до строки в БД	18
4 Backend: модели данных (<code>models/</code>)	24
5 Backend: Pydantic-схемы (<code>schemas/</code>)	31
6 Backend: репозитории (<code>repositories/</code>)	39
7 Backend: сервисы (<code>services/</code>)	45
8 Backend: роуты (<code>routes/</code>)	51
9 Backend: утилиты (<code>utils/</code>)	55
10 Аутентификация и безопасность	57
10.1 Два токена вместо одного	57
10.2 Роли и <code>require_role</code>	59
10.3 Разрешение области видимости по точке сети	59
10.4 Вход через VK ID (OAuth 2.1 + PKCE)	60
10.5 Защита от подбора пароля и лимиты запросов	61

10.6	Как cookie-авторизация выглядит со стороны фронтенда . . .	62
11	База данных и миграции	63
11.1	Alembic: схема БД – это история, а не снимок	63
11.2	Хронология: 15 миграций проекта	63
11.3	Двойное бронирование невозможно физически, а не по вежливой просьбе	66
11.4	backend/schema.sql – читаемый снимок, не источник истины	67
12	Тесты бэкенда	69
12.1	Два принципиально разных набора тестов	69
12.2	Каталог тестов	71
13	Frontend: точка входа и маршрутизация	74
13.1	main.js – где всё начинается	74
13.2	App.vue – корневой компонент	75
13.3	router/index.js – все маршруты в одном файле	75
14	Frontend: управление состоянием (Pinia stores)	78
14.1	stores/auth.js – кто сейчас залогинен	78
14.2	Остальные хранилища	79
15	Frontend: слой API (api/)	82
15.1	api/client.js – один axios на весь фронтенд	82
15.2	Остальные файлы api/	83
16	Frontend: компоненты	87
16.1	components/ui/ – переиспользуемые «кирпичики»	87
16.2	components/ – составные компоненты	91
16.3	composables/ – переиспользуемая логика	94
17	Frontend: страницы (views/)	96
17.1	BookingPage.vue – пошаговое бронирование	96
17.2	Остальные страницы	97
18	Frontend: тема оформления	102

19 Инфраструктура и деплой	104
19.1 Два Dockerfile	104
19.2 docker-compose.yml: пять сервисов плюс бэкапы	105
19.3 Caddy и nginx: кто кого проксирует	106
19.4 CI: восемь параллельных проверок	107
19.5 Обновление прод-версии: scripts/deploy.sh	108
20 Как запустить проект локально	110
20.1 Вариант 1: Docker, весь стек одной командой	110
20.2 Вариант 2: backend и frontend отдельно, без Docker	110
20.3 Письма и вход через VK ID на локальной машине	111
21 Куда смотреть дальше	112

1 Обзор проекта

Что делает система

CMS «Сайтама» автоматизирует запись клиентов в сеть парикмахерских и барбершопов: клиент сам выбирает мастера, услугу и свободное время без звонка администратору; у каждого мастера – свой прайс на одну и ту же услугу; система сама не даёт записать двух клиентов к одному мастеру на одно и то же время; клиентам приходят email-напоминания за 24 и за 2 часа до визита; администратор видит отчёты по выручке и загруженности без участия разработчика. Сеть может состоять из нескольких точек (salon) – этим управляет отдельная роль «владелец сети» (owner) поверх «администратора точки» (admin).

Роли пользователей

Почти весь код backend/ (роуты, сервисы) и немалая часть frontend/ (маршруты, условия в шаблонах) устроены вокруг четырёх ролей. Их стоит запомнить сразу – они будут встречаться в каждом разделе этого документа.

Таблица 1 – Роли пользователей системы

Роль	Что может
client (клиент)	Каталог мастеров и услуг, запись на свободное время, свои записи (просмотр/отмена), отзыв на завершённую запись, редактирование своего профиля. Роль по умолчанию при саморегистрации.
master (мастер)	Свой профиль и услуги (с индивидуальной ценой), своё расписание по дням недели, свои записи клиентов (подтвердить / завершить / отменить / перенести), отзывы о себе (только просмотр).

Роль	Что может
<code>admin</code> (администратор точки)	Все возможности мастера – плюс управление пользователями и мастерами <i>своей</i> точки, модерация отзывов своей точки, отчёты и статистика по своей точке. Обязательно привязан к одной точке (<code>salon_id</code>) – ограничение <code>ck_users_admin_requires_salon</code> в базе данных.
<code>owner</code> (владелец сети)	Всё то же, что и <code>admin</code> , но по всей сети сразу или по выбранной точке (переключатель в админке) – плюс то, что <code>admin</code> не может: создавать/закрывать точки сети, управлять общим каталогом услуг, назначать роль <code>admin</code> другим пользователям, редактировать контент и тему сайта.

Правило, которое стоит запомнить: `owner` – надмножество `admin` (в коде – функция `get_current_admin` в `backend/app/services/auth_service.py` явно пропускает и `owner` тоже). Отдельного дерева экранов или эндпоинтов «только для `owner`» почти нет – вместо этого одни и те же роуты и страницы проверяют *дополнительно*, что перед ними именно `owner`, там, где это нужно.

Технологический стек

Таблица 2 – Из чего состоит проект

Технология	Где используется
Python 3.14 + FastAPI	Backend: HTTP API, автоматическая документация <code>/api/docs</code> .
SQLAlchemy 2.0 (стиль <code>Mapped[...]</code>)	ORM – как Python-классы в <code>models/</code> превращаются в SQL-таблицы и запросы.

Технология	Где используется
Alembic	Версионизируемые миграции схемы БД (<code>backend/alembic/</code>).
Pydantic 2	Валидация входных данных и сериализация ответов (<code>schemas/</code>).
PostgreSQL 16	Единственная база данных; несколько бизнес-правил (например, запрет двойного бронирования) реализованы прямо в схеме БД, не только в коде.
Vue 3 (Composition API, <code><script setup></code>)	Frontend, без TypeScript.
Pinia	Хранилища состояния фронтенда (<code>frontend/src/stores/</code>).
Vue Router 4	Маршрутизация SPA, один файл <code>frontend/src/router/index.js</code> .
Tailwind CSS	Стили; цвета темы – CSS-переменные, переключаемые в рантайме, а не константы.
Vite	Сборка и dev-сервер фронтенда.
Docker Compose + Caddy	Развёртывание (раздел 19).

Слоистая архитектура backend

Backend (`backend/app/`) построен как строгая однонаправленная цепочка слоёв – каждый слой знает только о том, что находится под ним, и не знает о том, что выше. Рисунок 1 показывает эту цепочку вместе с тем, в какой папке искать код каждого слоя – к этому рисунку стоит возвращаться при чтении разделов 3–9.

Коротко, что делает каждый слой (подробно – в своём разделе):

- **routes/** (раздел 8) – принимает HTTP-запрос, проверяет роль пользователя через `Depends( . . . )`, вызывает ровно один метод нужного сервиса, отдаёт результат. Сам не решает бизнес-задач и не обращается к базе данных напрямую.

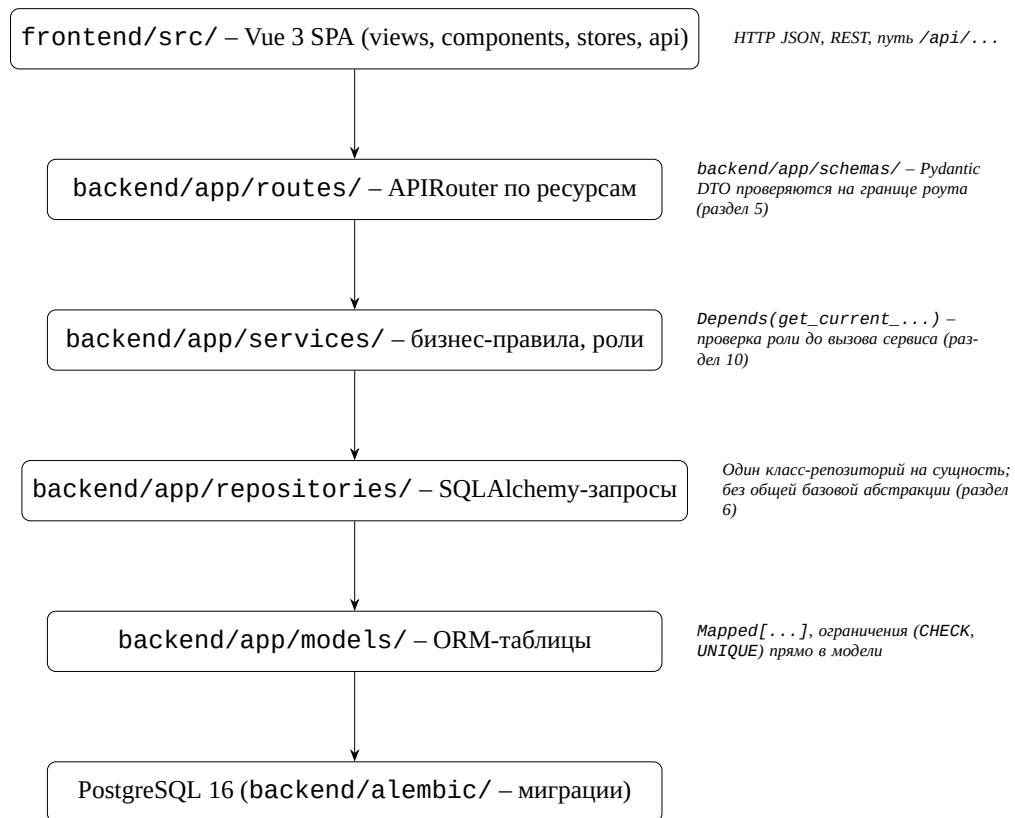


Рисунок 1 – Слоистая архитектура backend и где искать код каждого слоя

– **services/** (раздел 7) – вся бизнес-логика: «можно ли клиенту записаться на это время», «кто имеет право изменить эту запись», «как посчитать итоговую цену». Вызывает один или несколько репозиториев, ничего не знает про HTTP.

– **repositories/** (раздел 6) – единственное место, где пишутся SQLAlchemy-запросы (`select`, `insert` и т.д.). Один класс-репозиторий на одну сущность БД (пользователи, записи, точки сети...). Ничего не знает про бизнес-правила.

– **models/** (раздел 4) – Python-классы, описывающие таблицы БД (какие столбцы, какие ограничения).

– **schemas/** (раздел 5) – отдельная параллельная папка: Pydantic-классы, описывающие форму запроса/ответа HTTP API. Специально *не то же самое*, что модели – у клиента API должно быть можно поменять список полей, не трогая структуру таблицы, и наоборот.

Такое разделение называется **слоистой архитектурой** (layered architecture) с паттерном **Repository** (доступ к БД спрятан за классом с понятными методами вместо SQL-запросов, разбросанных по всему коду) и **DTO на границе** (Data Transfer Object – то, что видит клиент API, не обязано

совпадать с тем, как данные лежат в БД). Инъекция зависимостей (какой сервис получит какой репозиторий, какая сессия БД достанется запросу) устроена не через отдельный DI-контейнер, а через штатный механизм FastAPI – `Depends( . . . )` – он встретится почти в каждом примере кода этого документа.

Frontend: Vue 3 без TypeScript

Frontend (`frontend/src/`) вызывает backend по тем же правилам REST API, что и любой другой клиент. Пять папок стоит запомнить сразу (подробно – разделы 13–18):

- **views/** – страницы (одна на маршрут), собирают компоненты вместе.
- **components/** и **components/ui/** – переиспользуемые куски интерфейса; **ui/** – совсем мелкие «кирпичики» (кнопка, поле ввода), остальное – крупнее.
- **stores/** – Pinia-хранилища: состояние, общее для нескольких страниц (кто залогинен, список точек сети...).
- **api/** – тонкие обёртки над `axios` – по одному файлу на ресурс backend (`auth`, `masters`, `appointments...`), без логики внутри, только HTTP-вызовы.
- **router/index.js** – один файл, все маршруты приложения и проверки доступа к ним.

2 Структура репозитория верхнего уровня

Таблица 3 – беглый обзор того, что лежит в корне репозитория. Первые три строки – то, ради чего существует проект (backend/frontend/база данных как код); остальное – документация и инфраструктура вокруг них.

Таблица 3 – Каталоги и файлы верхнего уровня

Путь	Что это
backend/	FastAPI-приложение: код (app/), миграции (alembic/), тесты (tests/). Разделы 3–12 этого документа.
frontend/	Vue 3 SPA: код (src/), тесты (tests/). Разделы 13–18.
docker-compose.yml	Продовая сборка всего стека (Postgres + backend + frontend + Caddy + бэкапы). Раздел 19.
docker-compose.dev.yml	Дополнительный сервис для локальной разработки (Mailpit – локальный SMTP для писем). Раздел 19–20.
Caddyfile	Конфигурация Caddy – TLS-терминатор и security-заголовки. Раздел 19.
scripts/deploy.sh	Однокомандный скрипт обновления прод-сервера. Раздел 19.
.github/workflows/ci.yml	GitHub Actions – восемь параллельных проверок на каждый push/PR (раздел 12, раздел 19).
README.md	Обзор проекта, полный список возможностей по ролям, инструкция быстрого локального запуска.

Путь	Что это
<code>ROADMAP.md</code>	Большой рабочий документ: аудит проекта, план доработки, техническая спецификация функционала «сеть точек». Не продуктовый документ для внешнего читателя – рабочие заметки по ходу разработки.
<code>SECURITY.md</code>	Как сообщить об уязвимости (не через публичный Issue).
<code>new_issues.md</code>	Журнал разобранных замечаний по проекту (что было найдено, что и как исправлено) – история, а не открытые задачи.
<code>notes/</code>	Рабочие заметки по ходу разработки (анализ стека, подсказки для дальнейшей разработки с ИИ-ассистентом, черновик спецификации требований). Не входит в состав официальной документации проекта.
<code>final_report/</code> , <code>tech_zadanie/</code> , <code>user_manual/</code> , <code>admin_manual/</code>	Остальные документы проекта в формате LaTeX (см. раздел 21) – этот документ (<code>project_guide/</code>) пятый в этом ряду.
<code>graphify-out/</code>	Автоматически сгенерированный граф знаний по кодовой базе (инструмент <code>graphify</code>) – вспомогательный артефакт для навигации по коду, не документация проекта.
<code>claude_design_examples/</code> , <code>3d_models/</code>	Референсы дизайна и 3D-модель интерьера барбершопа (<code>.glb</code>), используемая компонентом <code>Hero3DRoom.vue</code> (раздел 16).
<code>backups/</code>	Точка монтирования для локальных бэкапов БД (см. <code>docker-compose.yml</code> , сервис <code>backup-local</code>).
<code>.env.example</code>	Шаблон переменных окружения для <code>docker-compose.yml</code> – какие переменные нужны и что они значат, с комментариями.

Путь	Что это
pyrightconfig.json	Конфигурация проверки типов Python (Pyright, режим <code>basic</code>) – только локально, не в CI.

3 Backend: архитектура и жизненный цикл приложения

3.1 Точка входа: `main.py` и `config.py`

Когда контейнер (или разработчик локально) запускает `uvicorn app.main:app`, выполняется файл `backend/app/main.py`. Он создаёт объект приложения FastAPI, подключает промежуточное ПО (CORS), регистрирует все роутеры и объявляет два служебных эндпоинта:

```
app = FastAPI(
    title=settings.app_name,
    debug=settings.debug,
    docs_url="/api/docs",
    redoc_url="/api/redoc",
    lifespan=lifespan,
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(setup_router)
app.include_router(auth_router)
app.include_router(services_router)
app.include_router(masters_router)
app.include_router(appointments_router)
app.include_router(admin_router)
app.include_router(reviews_router)
app.include_router(site_settings_router)
app.include_router(salons_router)

@app.get("/health")
def health(db: Session = Depends(get_db)):
    try:
        db.execute(text("SELECT 1"))
    except SQLAlchemyError as exc:
```

```

        raise HTTPException(status_code=503, detail="database
        ↪ unavailable") from exc
    return {"status": "ok"}

```

Три детали, на которые стоит обратить внимание сразу, потому что они типичны для всего проекта:

- `docs_url="/api/docs"` – FastAPI сам генерирует интерактивную документацию API (Swagger UI) из кода роутов и Pydantic-схем, без отдельного описания контракта вручную. Открыв локально запущенный backend на `http://localhost:8000/api/docs`, можно увидеть и вызвать вживую вообще все эндпоинты, описанные в разделах 4–9 этого документа.

- `lifespan=lifespan` – функция из `backend/app/scheduler.py` (раздел 3.2), которая запускается при старте и останавливается при выключении приложения – через неё поднимается фоновая задача рассылки напоминаний.

- `GET /health` делает настоящий запрос `SELECT 1` к базе данных, а не просто отвечает «процесс жив». Если БД недоступна, эндпоинт вернёт HTTP 503 – именно на этот путь смотрит `healthcheck` контейнера в `docker-compose.yml` (раздел 19), чтобы Docker понял, что контейнер нездоров, даже если сам процесс `uvicorn` ещё работает.

Все настраиваемые значения (строка подключения к БД, секретный ключ для JWT, лимиты запросов, SMTP и т. д.) собраны в одном месте – `backend/app/config.py`, классе `Settings` на базе `pydantic-settings`. Значения приходят из переменных окружения или файла `backend/.env`; для каждого поля в файле уже задан разумный дефолт для локальной разработки:

```

class Settings(BaseSettings):
    app_name: str = "Барбершоп «Сайтама»"
    debug: bool = True
    database_url: str = "postgresql://postgres:postgres@localhost:5432/barbershop"
    secret_key: str = _DEFAULT_SECRET_KEY
    access_token_expire_minutes: int = 15
    refresh_token_expire_days: int = 30
    ...
    model_config = SettingsConfigDict(env_file=".env")

```



```

@model_validator(mode="after")
def _no_default_secret_outside_debug(self) -> "Settings":
    if not self.debug and self.secret_key ==
        ↪ _DEFAULT_SECRET_KEY:
        raise RuntimeError(
            "SECRET_KEY не задан: при DEBUG=false задайте
            ↪ собственный "
            "SECRET_KEY через переменную окружения или
            ↪ backend/.env"
        )
    return self

```

Здесь стоит запомнить приём, который в проекте называют *fail-fast*: вне режима отладки (`DEBUG=false`, то есть в проде) приложение *отказывается запускаться*, если не задан собственный `SECRET_KEY` или `SETUP_TOKEN` – лучше увидеть ошибку запуска контейнера сразу, чем молча выдавать поддельные JWT-токены на дефолтном секретном ключе, известном каждому, кто читал исходный код.

3.2 Сессия БД и фоновые задачи: `database.py` и `scheduler.py`

`backend/app/database.py` создаёт SQLAlchemy-движок (engine) и определяет функцию `get_db()` – одну из самых часто встречающихся конструкций во всём backend-коде:

```

def get_db() -> Iterator[Session]:
    db = SessionLocal()
    try:
        yield db
        db.commit()
    except Exception:
        db.rollback()
        raise
    finally:
        db.close()

```

Это – FastAPI-Depends-зависимость, реализующая паттерн **Unit of Work**: на каждый HTTP-запрос создаётся своя сессия БД, запрос коммитится

один раз в самом конце, если всё прошло без исключений, и полностью откатывается, если где-то по пути (в роуте, в сервисе, в репозитории) было брошено исключение – в том числе `HTTPException`. Благодаря этому многошаговые операции (например, смена роли пользователя и деактивация его профиля мастера разом) либо выполняются целиком, либо не выполняются вовсе – никогда наполовину. В коде роута эта зависимость выглядит так: `db: Session = Depends(get_db)` – этот параметр встретится почти в каждом обработчике роута в разделе 8.

`backend/app/scheduler.py` отвечает за единственную фоновую задачу приложения – рассылку напоминаний о записи. Ни отдельного воркера, ни очереди задач (Celery/Redis) в проекте нет: вместо этого внутри того же процесса backend работает бесконечный `asyncio`-цикл, запускаемый и останавливаемый вместе с самим приложением через `lifespan`:

```
async def _run_reminder_pass() -> None:
    db = SessionLocal()
    try:
        await asyncio.to_thread(ReminderService(db).send_due_rema_
            ↪ inders)
    finally:
        db.close()
```

```
async def _reminder_loop() -> None:
    while True:
        try:
            await _run_reminder_pass()
        except Exception:
            logger.exception("Не удалось выполнить проход
                ↪  напоминаний о записях")
        await asyncio.sleep(settings.reminder_poll_interval_seco_
            ↪ nds)
```

```
@contextlib.asynccontextmanager
async def lifespan(app: FastAPI) -> AsyncIterator[None]:
    task = asyncio.create_task(_reminder_loop())
    try:
        yield
    finally:
```

```
task.cancel()
```

`asyncio.to_thread(...)` здесь важен: `ReminderService` внутри делает обычные синхронные вызовы `SQLAlchemy` и `SMTP`, которые сами по себе заблокировали бы весь событийный цикл (`event loop`) – тот же самый цикл, который в этот момент должен успевать обслуживать HTTP-запросы пользователей сайта. Уводя синхронный код в отдельный поток, цикл остаётся свободным. Раз в `settings.reminder_poll_interval_seconds` секунд (по умолчанию 300, то есть 5 минут) цикл спрашивает у `ReminderService`, кому пора отправить письмо – подробнее в разделе 7 (`services/reminder_service.py`). Такое решение осознанно упрощённое: оно не масштабируется на несколько работающих одновременно копий `backend` (каждая запустила бы свой независимый цикл и письма продублировались бы) – проект и рассчитан ровно на один работающий экземпляр контейнера `backend` (то же ограничение разделяют защита от перебора паролей и общие лимиты запросов, раздел 10).

3.3 Слои на примере одной записи: от HTTP-запроса до строки в БД

Самый быстрый способ понять, как слои вызывают друг друга – пройти путь одного HTTP-запроса целиком, слой за слоем. Возьмём самый частый и самый важный сценарий в системе: клиент нажимает «Записаться» на странице мастера, и браузер отправляет `POST /api/appointments`.

Шаг 1 – роут принимает запрос `backend/app/routes/appointments.py`:

```
@router.post("", response_model=AppointmentResponse,
              status_code=status.HTTP_201_CREATED)
@limiter.limit(settings.rate_limit_booking)
def create_appointment(
    request: Request,
    data: AppointmentCreate,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_client),
):
    if not current_user.email:
```

```

        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
            detail="Укажите email в профиле,
            чтобы оформить запись")
    return AppointmentService(db).create(current_user.id, data)

```

Роут сам почти ничего не делает – и это правильно, так и задумано. `Depends(get_current_client)` уже к этому моменту проверил JWT-токен из cookie и роль пользователя (раздел 10) – если бы токена не было или роль не подходила, до этой строчки код вообще не дошёл бы, FastAPI сам вернул бы 401 или 403. `data: AppointmentCreate` – тело запроса уже провалидировано и превращено в Python-объект (шаг 2 ниже) до того, как выполнилась первая строчка функции. Единственная бизнес-проверка прямо в роуте (нужен email в профиле) – узкое исключение, оставленное здесь, а не в сервисе, потому что она специфична именно для HTTP-сценария записи через VK ID без email (раздел 10). Вся остальная работа – `AppointmentService(db).create(...)`, один вызов.

Шаг 2 – Pydantic-схема проверяет форму запроса `backend/app/schemas/appointment.py`:

```

class AppointmentCreate(BaseModel):
    master_id: Annotated[UUID, Field(description="UUID
        ↳ мастера")]
    service_id: Annotated[UUID, Field(description="UUID услуги")]
    start_time: Annotated[AwareDatetime, Field(
        description="Желаемое время начала (обязательно с
        ↳ таймзоной; нормализуется в UTC)")]

    @field_validator("start_time")
    @classmethod
    def _normalize_to_utc(cls, v: datetime) -> datetime:
        return v.astimezone(timezone.utc)

```

Клиент присылает всего три поля – мастер, услуга, желаемое время начала. `end_time` и итоговая цена клиенту не доверяются вовсе: их вычисляет сервер (шаг 3). Валидатор `_normalize_to_utc` переводит присланное время в UTC *на границе*, один раз – вся дальнейшая логика расписания и сло-

тов (и на backend, и на frontend) может спокойно считать, что работает в UTC, не заботясь о часовом поясе браузера клиента.

Шаг 3 – сервис проверяет бизнес-правила backend/app/services/appointment_service.py, метод AppointmentService.create() – самая длинная и самая показательная функция всего проекта, стоит прочитать её целиком в файле; здесь – сокращённая версия с сохранёнными номерами шагов из комментариев самого кода:

```
def create(self, client_id: UUID, data: AppointmentCreate) ->
    AppointmentResponse:
    # 0. Лимит одновременных незавершённых записей клиента
    active_count =
        ↪ self.appointment_repo.count_active_for_client(client_id)
    if active_count >=
        ↪ settings.max_active_appointments_per_client:
        raise HTTPException(status_code=400, detail="Слишком
            ↪ много активных записей...")

    # 1. Мастер и услуга существуют и активны
    master = self.master_repo.get_by_id(data.master_id)
    if not master or not master.is_active:
        raise HTTPException(status_code=404, detail="Мастер не
            ↪ найден или неактивен")
    if master.user_id == client_id:
        raise HTTPException(status_code=400, detail="Нельзя
            ↪ записаться к себе")
    service = self.service_repo.get_by_id(data.service_id)
    ...

    # 2. Мастер оказывает именно эту услугу?
    ms = self.master_repo.get_master_service(data.master_id,
        ↪ data.service_id)
    if not ms:
        raise HTTPException(status_code=400, detail="Мастер не
            ↪ оказывает данную услугу")

    # 3. Вычисляем end_time и итоговую цену (Decimal, не float --
        ↪ не теряет точность)
    end_time = data.start_time +
        ↪ timedelta(minutes=service.duration_min)
```

```

final_price = (
    ms.price_override
    if ms.price_override is not None
    else (service.price *
        ↪ master.coefficient).quantize(Decimal("0.01"),
        ↪ rounding=ROUND_HALF_UP)
)

# 4. Слот попадает в рабочее расписание мастера и в часы
    ↪ работы точки?
self._validate_within_schedule(data.master_id,
    ↪ master.salon_id, data.start_time, end_time)

# 5. Нет ли уже другой записи на это время? (предварительная
    ↪ проверка)
overlap =
    ↪ self.appointment_repo.get_overlapping(data.master_id,
    ↪ data.start_time, end_time)
if overlap:
    raise HTTPException(status_code=409, detail="Выбранное
        ↪ время уже занято")

# 6. Сохраняем. От гонки двух одновременных запросов защищает
    ↪ не эта
# проверка выше, а EXCLUDE-ограничение в самой базе данных
    ↪ (раздел 11).
try:
    appointment = self.appointment_repo.create(
        client_id, data, end_time, final_price,
        ↪ master.salon_id
    )
except IntegrityError as exc:
    self.db.rollback()
    raise HTTPException(status_code=409, detail="Выбранное
        ↪ время уже занято") from exc
return AppointmentResponse.model_validate(
    self.appointment_repo.get_by_id(appointment.id)
)

```

Шаг 6 – ключевое место всей архитектуры проекта, и стоит остановиться на нём подробнее. Проверка «нет ли пересечения» на шаге 5 *сама по себе не защищает* от двойного бронирования: если два клиента отправят запрос

на один и тот же слот одновременно, оба запроса могут пройти проверку на шаге 5 до того, как первый из них успеет сохранить свою запись – это называется состоянием гонки (race condition). Шаг 5 существует только ради понятной ошибки в обычном (не гоночном) случае. Настоящая защита – на уровне самой базы данных: если сохранить одновременно два пересекающихся интервала у одного мастера всё-таки попытаются, PostgreSQL физически откажет второму запросу через ограничение EXCLUDE USING gist (раздел 11) – и именно поэтому вокруг `appointment_repo.create(...)` стоит `try/except IntegrityError`, превращающий отказ базы данных в понятный клиенту ответ 409, а не в необработанную ошибку 500.

Шаг 4 – репозиторий сохраняет запись `backend/app/repositories/appointment_repository.py`:

```
def create(self, client_id: uuid.UUID, data: AppointmentCreate,
            end_time: datetime, final_price: Decimal, salon_id:
            → uuid.UUID) -> Appointment:
    appointment = Appointment(
        client_id=client_id,
        master_id=data.master_id,
        service_id=data.service_id,
        salon_id=salon_id,
        start_time=data.start_time,
        end_time=end_time,
        final_price=final_price,
        status=AppointmentStatus.pending,
    )
    self.db.add(appointment)
    self.db.flush()
    self.db.refresh(appointment)
    return appointment
```

Репозиторий – единственное место во всём проекте, которое строит SQLAlchemy-объекты и обращается к `self.db`. Никакой бизнес-логики здесь уже нет: всё (цена, `end_time`, `salon_id`) уже вычислено сервисом на шаге 3 и передано сюда готовым. Обратите внимание на `self.db.flush()` вместо `self.db.commit()` – репозиторий отправляет изменения в БД (чтобы, например, `IntegrityError` от ограничения EXCLUDE произошёл

именно здесь и его можно было поймать), но не завершает транзакцию – это исключительно обязанность `get_db()` из раздела 3.2, и только когда весь запрос отработал успешно.

Шаг 5 – ответ сериализуется обратно в JSON `AppointmentResponse.model_validate(...)` в конце шага 3 – это переход из мира ORM обратно в мир DTO: Pydantic-схема `AppointmentResponse` (`backend/app/schemas/appointment.py`) сама решает, какие поля `Appointment` видит клиент API, и сериализует их в JSON. «Сырые» ORM-объекты никогда не покидают `backend` через HTTP – этому правилу подчиняется вообще весь проект, не только этот эндпоинт.

Итог: один HTTP-запрос прошёл через все пять слоёв ровно по стрелкам Рисунка 1 – ни разу в обратную сторону и ни разу «в обход». Эта же схема без изменений повторяется для *любого* другого эндпоинта проекта – разница только в конкретных бизнес-правилах внутри сервиса. Держите этот пример перед глазами, читая разделы 4–9: они описывают те же пять слоёв, но уже для остальных 12 сущностей системы (пользователи, точки сети, услуги, от-зывы...).

4 Backend: модели данных (models/)

Каждый файл в `backend/app/models/` – один Python-класс, описывающий одну таблицу БД через SQLAlchemy 2.0 (стиль `Mapped[...]`). Три общих «примеси» переиспользуются почти везде – смотрите первую запись ниже (`mixins.py`) перед остальными: не пришлось бы гадать, почему у большинства моделей нет собственного объявления `id/created_at`.

backend/app/models/mixins.py Общие «примеси» (`mixins`) для переиспользования одинаковых столбцов между моделями – чтобы не дублировать код первичного ключа, отметки времени создания и мягкого удаления в каждом файле модели отдельно. Почти все модели проекта наследуются от одного или нескольких из этих трёх классов.

```
class UUIDPrimaryKeyMixin    # столбец id: UUID (генерируется в
    ↪ Python, uuid.uuid4)
class TimestampMixin         # столбец created_at: datetime
    ↪ (server_default=func.now())
class SoftDeleteMixin        # столбец deleted_at: datetime | None
SoftDeleteMixin.is_deleted(self) -> bool    # property: True, если
    ↪ deleted_at заполнен
```

Особенности: `SoftDeleteMixin` реализует паттерн мягкого удаления – запись помечается удалённой, но физически остаётся в БД, так как на неё могут ссылаться другие таблицы через `onDelete=RESTRICT` (например, записи на приём ссылаются на пользователя).

backend/app/models/user.py Пользователь (`User`) – центральная сущность системы: один и тот же класс описывает клиента, мастера, администратора точки и владельца сети, различаемых полем `role`. Самый связанный класс во всём проекте – полный разбор кода уже приведён в разделе 3.3 (шаг 3) и в разделе 10 (аутентификация); здесь – только сводка.

```
class User(Base, UUIDPrimaryKeyMixin, TimestampMixin,
    ↪ SoftDeleteMixin)
User.email_verified(self) -> bool    # property
User.__repr__(self) -> str
```

Особенности: партиционные уникальные индексы на `email/phone/vk_user_id` действуют только среди «живых» строк (`deleted_at IS NULL`); ограничение `ck_users_admin_requires_salon` в БД не даёт роли `admin` существовать без привязанной точки; `token_version` инкрементируется при `logout`/смене пароля/блокировке – так отзываются ранее выданные JWT без хранения отдельного «чёрного списка» токенов.

backend/app/models/enums.py Два перечисления (Python `enum`), используемых по всему проекту – и на `backend`, и (теми же строковыми значениями) на `frontend`.

```
class UserRole(str, enum.Enum):
    client = "client"
    master = "master"
    admin  = "admin"
    owner  = "owner"

class AppointmentStatus(str, enum.Enum):
    pending   = "pending"
    confirmed = "confirmed"
    cancelled = "cancelled"
    done      = "done"
```

Особенности: допустимые переходы между значениями `AppointmentStatus` проверяются не здесь, а в `AppointmentService` (словарь `ALLOWED_TRANSITIONS`, раздел 3.3 и раздел 7) – сама модель ничего не знает про машину состояний.

backend/app/models/appointment.py Запись клиента к мастеру на конкретную услугу – уже подробно разобрана в разделе 3.3 целиком, включая защиту от двойного бронирования (`EXCLUDE USING gist`, раздел 11). Кратко:

```
class Appointment(Base, UUIDPrimaryKeyMixin, TimestampMixin)
Appointment.review_id(self) -> uuid.UUID | None      # property
Appointment.client_name(self) -> str                  # property
↪ (денормализация для списков)
Appointment.master_name(self) -> str                  # property
Appointment.service_name(self) -> str                # property
Appointment.__repr__(self) -> str
```

Особенности: `salon_id` денормализован из `master.salon_id` на момент создания записи – перевод мастера в другую точку задним числом не переносит его старые записи; так же «заморожена» `final_price` – изменение прайса не меняет цену уже созданных записей.

backend/app/models/master.py Определяет мастера (парикмахера/барбера) – сотрудника, оказывающего услуги в конкретной точке сети (`salon`), и связь мастера с услугами, которые он выполняет. `Master` связан с `User` (учётной записью для входа) и с `Salon`: один мастер физически работает только в одной точке. `MasterService` – связующая таблица «многие-ко-многим» между мастерами и услугами, позволяющая задать мастеру индивидуальную цену на конкретную услугу.

```
class Master(Base, UUIDPrimaryKeyMixin, TimestampMixin,
    ↪ SoftDeleteMixin)
Master.first_name(self) -> str          # property, делегирует к
    ↪ self.user.first_name
Master.last_name(self) -> str          # property, делегирует к
    ↪ self.user.last_name
Master.salon_name(self) -> str         # property, делегирует к
    ↪ self.salon.name
Master.__repr__(self) -> str

class MasterService(Base)               # связь N:M «мастер <->
    ↪ услуга»
MasterService.__repr__(self) -> str
```

Особенности: частичный уникальный индекс `uq_masters_user_id_active` действует только при `deleted_at IS NULL` – после мягкого удаления профиля мастера тот же пользователь может получить новый активный профиль. Итоговая цена услуги мастера = `services.price * masters.coefficient`, если в `master_services.price_override` не задано конкретное значение.

backend/app/models/salon.py Точка сети барбершопов (конкретный физический адрес/филиал), к которой привязаны мастера и записи на услуги. Хранит название, адрес, телефон, часы работы и фото. Проект поддерживает сеть из нескольких точек, поэтому мастер и запись обязательно

ссылаются на конкретный `salon`.

```
class Salon(Base, UUIDPrimaryKeyMixin, TimestampMixin)
Salon.__repr__(self) -> str
```

Особенности: не использует `SoftDeleteMixin` – точка не удаляется физически (мастера/записи ссылаются на неё с `ondelete=RESTRICT`), а «закрывается» через `is_active=false`; `slug` генерируется сервером из `name` (транслитерация + `slugify`, раздел 9) и не редактируется вручную.

backend/app/models/service.py Услуга, которую оказывает барбершоп (например, стрижка или бритьё) – с базовой ценой и длительностью в минутах. Услуга входит в общий каталог сети (не привязана к конкретной точке) и может выполняться разными мастерами через связь `MasterService`.

```
class Service(Base, UUIDPrimaryKeyMixin, SoftDeleteMixin)
Service.__repr__(self) -> str
```

Особенности: наследует `SoftDeleteMixin`, но не `TimestampMixin` (нет `created_at`); `is_active` (публикация в каталоге) и `deleted_at` (мягкое удаление) – два независимых механизма скрытия услуги: первый временно прячет её из каталога, второй убирает полностью.

backend/app/models/schedule.py Рабочее расписание конкретного мастера по дням недели – во сколько мастер начинает и заканчивает работу и является ли день рабочим. У каждого мастера не может быть больше одной строки расписания на день недели (0 = понедельник, ..., 6 = воскресенье).

```
class Schedule(Base, UUIDPrimaryKeyMixin, TimestampMixin)
Schedule.__repr__(self) -> str
```

Особенности: `UniqueConstraint` на (`master_id`, `day_of_week`) не даёт создать два расписания на один день недели для одного мастера; CHECK-ограничения в БД требуют `end_time > start_time` и `day_of_week` в диапазоне 0–6.

backend/app/models/review.py Отзыв клиента о завершённой записи (`appointment`) – оценка от 1 до 5 и необязательный текстовый комментарий. Каждая запись на услугу может получить максимум один отзыв.

Отзыв публикуется сразу после создания, но администратор может скрыть его через `is_published`, не удаляя из базы.

```
class Review(Base, UUIDPrimaryKeyMixin, TimestampMixin)
Review.client_name(self) -> str      # property
Review.master_name(self) -> str      # property
Review.service_name(self) -> str     # property
Review.__repr__(self) -> str
```

Особенности: `master_id` и `service_id` денормализованы из связанной записи `appointment` в момент создания отзыва – это позволяет выбирать отзывы конкретного мастера/услуги без join’а через `appointments`; уникальное ограничение на `appointment_id` гарантирует «один отзыв на одну запись».

backend/app/models/session.py Сессия входа пользователя – по сути refresh-токен, позволяющий получить новый access-токен (JWT) без повторного ввода пароля. В БД хранится только хеш токена; сам токен живёт в `httpOnly-cookie` на устройстве пользователя. У одного пользователя может быть несколько активных сессий (по одной на устройство).

```
class Session(Base, UUIDPrimaryKeyMixin, TimestampMixin)
Session.__repr__(self) -> str
```

Особенности: `logout` удаляет только одну сессию (текущее устройство), а события безопасности (смена/сброс пароля, блокировка, мягкое удаление пользователя) удаляют разом все сессии пользователя синхронно с увеличением `token_version` – иначе access-токен был бы отозван, а refresh-токен продолжал бы тихо выдавать новые рабочие access-токены (подробнее – раздел 10).

backend/app/models/password_reset_token.py Таблица токенов для восстановления забытого пароля. Пользователь запрашивает сброс, получает ссылку с токеном, а в БД хранится только SHA-256-хеш этого токена вместе со сроком действия и отметкой об использовании.

```
class PasswordResetToken(Base, UUIDPrimaryKeyMixin,
    ↪ TimestampMixin)
PasswordResetToken.__repr__(self) -> str
```

Особенности: в БД хранится только хеш токена, никогда сам токен – тот живёт лишь в ссылке, отправленной пользователю письмом.

backend/app/models/email_verification_token.py Таблица для подтверждения email-адреса пользователя после регистрации. Устроена почти так же, как `password_reset_token.py`, только для сценария «подтвердите почту».

```
class EmailVerificationToken(Base, UUIDPrimaryKeyMixin,
    ↳ TimestampMixin)
EmailVerificationToken.__repr__(self) -> str
```

Особенности: тот же приём, что и у `PasswordResetToken` – в БД лежит только SHA-256-хеш токена.

backend/app/models/login_attempt.py Таблица аудита попыток входа в систему – каждая попытка логина (успешная или нет) записывается сюда с email, IP-адресом и результатом. Используется для требований безопасности – отслеживания подозрительной активности и защиты от подбора пароля. Если введённый email не принадлежит ни одному зарегистрированному пользователю, попытка всё равно логируется.

```
class LoginAttempt(Base, UUIDPrimaryKeyMixin, TimestampMixin)
LoginAttempt.__repr__(self) -> str
```

Особенности: `user_id` – внешний ключ с `ondelete="SET NULL"` и допускает NULL, то есть попытка входа сохраняется в логе, даже если email не совпал ни с одним пользователем или пользователь позже был удалён.

backend/app/models/site_settings.py Глобальные настройки и редактируемый контент сайта (тексты на главной странице, тема оформления, SEO и т. д.) – то, что владелец сети может менять через CMS-панель без правки кода. Таблица всегда содержит ровно одну строку (singleton), которая лениво создаётся сервисом при первом обращении.

```
class SiteSettings(Base, UUIDPrimaryKeyMixin, TimestampMixin)
```

Особенности: весь редактируемый контент хранится в одном JSONB-столбце `content` (структура описана в `schemas/site_settings.py`,

класс `SiteContent`), а не в отдельных колонках – так добавление нового редактируемого поля не требует новой миграции БД.

5 Backend: Pydantic-схемы (schemas/)

Каждый файл в `backend/app/schemas/` описывает форму HTTP-запроса/ответа – то, что видит клиент API. Схемы намеренно живут в отдельной от `models/` папке (раздел 3, паттерн DTO): у одной сущности БД может быть несколько разных схем-ответов (полная версия для владельца данных, урезанная – для посторонних).

`backend/app/schemas/fields.py` Общий модуль переиспользуемых типов полей (через `Annotated`) для всех Pydantic-схем проекта – email, пароль, телефон, цвет темы, денежные суммы и рейтинг. Централизует правила валидации (например, ограничение длины пароля под `bcrypt` или нормализацию email в нижний регистр), чтобы не дублировать их в каждой схеме. Читать этот файл стоит раньше остальных схем – его типы встречаются почти в каждой из них.

```
_fits_bcrypt(v: str) -> str          # валидатор: пароль не
↳ длиннее 72 байт в UTF-8
NameStr          # Annotated[str]: имя/фамилия, 1-100 символов
NormalizedEmailStr # Annotated[EmailStr]: email приводится к
↳ нижнему регистру
HexColorStr      # Annotated[str]: строго формат #RRGGBB
PhoneStr         # Annotated[str]: телефон, 5-20 символов
PasswordStr      # Annotated[str]: пароль 8-72 символа +
↳ проверка _fits_bcrypt
PositiveMoney    # Annotated[Decimal]: сумма > 0, сериализуется
↳ в JSON как число
NonNegativeMoney # Annotated[Decimal]: сумма >= 0
MoneyOut         # Annotated[Decimal]: для ответов API, без
↳ ограничений на ввод
RatingInt        # Annotated[int]: оценка от 1 до 5
```

Особенности: `bcrypt` (начиная с версии 5.x) выбрасывает `ValueError` на пароль длиннее 72 байт – без верхней границы в `PasswordStr` длинный пароль ронял бы регистрацию/сброс пароля ошибкой 500 вместо понятной 400; денежные суммы в Python – `Decimal` (не `float`, чтобы не терять точность), но в JSON отдаются обычным числом, а не строкой.

backend/app/schemas/appointment.py DTO для записи на приём – разобраны целиком в разделе 3.3 (AppointmentCreate на шаге 2). Полный список классов:

```
class AppointmentCreate(BaseModel)           # что присылает
    ↪ клиент (см. раздел 3.3)
class AppointmentStatusUpdate(BaseModel)      # смена статуса
    ↪ мастером/админом
class AppointmentReschedule(BaseModel)        # перенос записи на
    ↪ другое время
class AppointmentBriefResponse(BaseModel)     # краткий ответ для
    ↪ списков
class AppointmentResponse(BaseModel)          # полный ответ для
    ↪ детальной страницы
class SlotResponse(BaseModel)                # один свободный
    ↪ временной слот
class SlotListResponse(BaseModel)            # список слотов на
    ↪ дату
```

Особенности: AppointmentReschedule валидирует только новое время – мастер и услуга не меняются, end_time пересчитывается на сервере из длительности той же услуги.

backend/app/schemas/user.py Pydantic-схемы для пользователя системы (клиент, мастер или администратор) – базовые поля профиля, саморегистрация, создание/редактирование пользователя администратором и три варианта ответа API с разным уровнем раскрытия персональных данных.

```
class UserBase(BaseModel)                   # общие поля: email,
    ↪ first_name, last_name, phone
class UserCreate(UserBase)                  # тело запроса
    ↪ саморегистрации (+ password)
class AdminUserCreate(UserCreate)           # создание пользователя
    ↪ администратором (+ role)
class UserUpdate(BaseModel)                 # самостоятельное
    ↪ обновление профиля
class AdminUserUpdate(BaseModel)            # редактирование
    ↪ пользователя админом (+ new_password)
class UserResponse(BaseModel)               # полный ответ (для
    ↪ владельца аккаунта/админа)
class UserPublicResponse(BaseModel)         # публичная проекция
    ↪ (только имя, без контактов)
```

```
class ClientBriefResponse(BaseModel)    # клиент в контексте
↳ записи (имя + телефон)
```

Особенности: email в `UserUpdate` необязателен – актуально для клиентов, зарегистрированных через VK без email (VK ID не всегда его отдаёт), им предлагается указать email при оформлении первой записи (раздел 3.3); `ClientBriefResponse` и `UserPublicResponse` – намеренно урезанные проекции, чтобы не раскрывать email/роль/`is_blocked` посторонним.

backend/app/schemas/auth.py Pydantic-схемы для эндпоинтов авторизации – вход по email/паролю, ответ с JWT-токеном, запрос и подтверждение восстановления пароля, подтверждение email.

```
class LoginRequest(BaseModel)           # тело запроса на
↳ вход (email + password)
class TokenResponse(BaseModel)          # ответ с
↳ access-токеном и данными пользователя
class PasswordResetRequest(BaseModel)   # тело запроса на
↳ восстановление пароля
class PasswordResetConfirm(BaseModel)   # подтверждение
↳ сброса пароля (token + new_password)
class EmailVerificationConfirm(BaseModel) # подтверждение email
↳ (token)
```

backend/app/schemas/master.py Pydantic-схемы, описывающие мастера и его услуги в разных контекстах API – от краткой карточки в каталоге до полного профиля с контактами (для самого мастера/администратора) и данных для обновления профиля или прайса.

```
class MasterServiceResponse(BaseModel)  # услуга в контексте
↳ мастера (с итоговой ценой)
class MasterBriefResponse(BaseModel)    # краткая карточка
↳ мастера для списков/каталога
class MasterResponse(BaseModel)         # полный профиль (для
↳ самого мастера/администратора)
class MasterPublicResponse(BaseModel)   # публичный профиль (без
↳ email/телефона)
class MasterUpdate(BaseModel)           # обновление профиля
↳ мастера
class MasterServiceCreate(BaseModel)    # добавление услуги
↳ мастеру
```

```
class MasterPhotoUpdate(BaseModel)          # смена/очистка фото
    ↪ мастера
```

Особенности: `MasterPublicResponse.rating` агрегируется на бэкенде по всем опубликованным отзывам – фронтенд не должен пересчитывать его по одной странице выдачи; `MasterUpdate.salon_id` позволяет переводить мастера между точками сети (разрешено только владельцу сети).

backend/app/schemas/service.py Pydantic-схемы для услуги барбершопа – базовые поля, создание, обновление и ответ API.

```
class ServiceBase(BaseModel)          # общие поля: name,
    ↪ description, price, duration_min
class ServiceCreate(ServiceBase)      # тело запроса на создание
    ↪ услуги
class ServiceUpdate(BaseModel)        # частичное обновление услуги
class ServiceResponse(BaseModel)      # ответ API с данными услуги
```

backend/app/schemas/schedule.py Pydantic-схемы для рабочего расписания мастера – создание и обновление записи расписания на один день недели, а также ответ API.

```
class ScheduleCreate(BaseModel)          #
    ↪ день_недели, start_time, end_time, is_working
ScheduleCreate.end_after_start(self)      #
    ↪ model_validator(mode="after")
class ScheduleUpdate(BaseModel)          # частичное
    ↪ обновление
ScheduleUpdate.end_after_start(self)      #
    ↪ model_validator(mode="after")
class ScheduleResponse(BaseModel)
```

Особенности: `ScheduleUpdate.end_after_start` сравнивает поля через `is not None`, а не через `truthiness` – время `00:00` (полночь) в Python ложно (`falsy`), и проверка вида «если значение задано» через простое `if self.start_time` молча пропускала бы валидацию именно для полуночи.

backend/app/schemas/salon.py Pydantic-схемы для точки сети (`salon`) – базовые поля, создание, обновление и два варианта ответа

API (полный и краткий, для встраивания в другие ответы вроде профиля мастера).

```
class SalonBase(BaseModel)          # name, address, phone,
    ↪ open_time, close_time, photo_url
class SalonCreate(SalonBase)
SalonCreate.close_after_open(self)  #
    ↪ model_validator(mode="after")
class SalonUpdate(BaseModel)
SalonUpdate.close_after_open(self)  #
    ↪ model_validator(mode="after")
class SalonResponse(BaseModel)
class SalonBriefResponse(BaseModel) # краткая версия для
    ↪ встраивания в другие схемы
```

Особенности: slug сознательно отсутствует в SalonCreate/SalonUpdate – генерируется сервером из name (раздел 9); SalonUpdate.close_after_open проверяет пару open_time/close_time только когда переданы оба поля сразу.

backend/app/schemas/review.py Pydantic-схемы для создания отзыва клиентом, модерации отзыва администратором (публикация/скрытие) и ответа API с данными отзыва.

```
_validate_comment_content(v: str | None) -> str | None  #
    ↪ валидатор: мат/ссылки
class ReviewCreate(BaseModel)          # appointment_id, rating,
    ↪ comment
class ReviewModerate(BaseModel)        # is_published
class ReviewResponse(BaseModel)
```

Особенности: отзыв публикуется сразу без модерации (is_published – лишь пост-фактум скрытие администратором), поэтому валидатор _validate_comment_content обязан целиком отклонить запрос (raise ValueError) при мате или ссылках, а не просто пометить его на проверку – использует utils/content_filter.py (раздел 9).

backend/app/schemas/pagination.py Универсальные вспомогательные классы для постраничной выдачи списков (пагинации) – используются во всех эндпоинтах, отдающих список записей.

```

class PageParams                                # FastAPI-зависимость
    ↪ параметров из query-строки
PageParams.__init__(self, page: int = 1, page_size: int = 20) ->
    ↪ None
PageParams.offset(self) -> int                  # property
class PageResponse(BaseModel, Generic[T])      # обёртка:
    ↪ items/total/page/page_size
PageResponse.total_pages(self) -> int          # computed_field

```

Особенности: `page_size` ограничен диапазоном 1–20; `total_pages` вычисляется приёмом «деление с округлением вверх без импорта `math`» – `-(-total // page_size)`.

backend/app/schemas/admin_stats.py Pydantic-схемы для ответа административной панели со сводной статистикой – общее число пользователей, клиентов, мастеров, услуг, записей за месяц, выручка за месяц и график регистраций за последние 30 дней.

```

class DailyCount(BaseModel)                    # пара «дата + количество»
    ↪ (для графика регистраций)
class AdminStatsResponse(BaseModel)           # сводная статистика для
    ↪ главного экрана админки

```

backend/app/schemas/report.py Pydantic-схемы для отчёта по бизнес-показателям (выручка, число записей, средний чек, разбивка по дням/услугам/мастерам) за выбранный период.

```

class DailyRevenue(BaseModel)                 # выручка за один день
class ServiceReportRow(BaseModel)             # строка отчёта по услуге
class MasterReportRow(BaseModel)              # строка отчёта по мастеру
class ReportResponse(BaseModel)               # полный отчёт: итоги +
    ↪ разбивки

```

backend/app/schemas/setup.py Pydantic-схемы для первоначальной настройки системы при первом запуске (bootstrap) – создание первого аккаунта-владельца сети и первой точки, а также проверка, завершена ли установка.

```

class SetupStatusResponse(BaseModel)          # завершена ли установка,
    ↪ требуется ли setup_token

```

```
class SetupRequest(BaseModel)          # owner, salon,
    ↪ site_content, setup_token
```

Особенности: первый созданный пользователь получает роль `owner` (не `admin`) – роль `admin` привязана к конкретной точке сети, которой на первом запуске ещё не существует.

backend/app/schemas/site_settings.py Pydantic-схемы для всего редактируемого контента сайта (CMS) – тексты и медиа шапки, SEO-теги, главного экрана, блока преимуществ, каталога услуг, блока мастеров, призыва к действию, подвала, а также цветовая тема и шрифт оформления. `SiteContent` – корневая схема, которая целиком читается и записывается через админ-панель и хранится как JSONB в модели `SiteSettings`.

```
class HeaderContent(BaseModel)        # текст шапки (название
    ↪ бренда, слоган)
class SeoContent(BaseModel)           # SEO-теги (title,
    ↪ description, favicon_url)
class HeroContent(BaseModel)          # содержимое главного экрана
class FeaturesContent(BaseModel)      # блок «почему мы»
class ServicesContent(BaseModel)      # заголовок/примечание для
    ↪ блока каталога услуг
class MastersContent(BaseModel)       # заголовок для блока мастеров
class CtaContent(BaseModel)           # блок призыва к действию
class SocialLink(BaseModel)           # одна ссылка на
    ↪ соцсеть/ресурс
class FooterContent(BaseModel)        # содержимое подвала сайта
class ThemeColors(BaseModel)          # набор цветов темы
class ThemeContent(BaseModel)         # тема оформления целиком
class SiteContent(BaseModel)          # корневая схема всего
    ↪ контента сайта
```

Особенности: значения по умолчанию во всех классах повторяют текущие захардкоженные тексты фронтенда – если владелец ничего не сохранял, сайт выглядит как сейчас; `FooterContent` сознательно не содержит `address/hours` – это атрибуты конкретной точки сети (`Salon`), а не бренда в целом; `SiteContent` не запрещает лишние поля, поэтому устаревшие ключи в уже сохранённом JSONB просто игнорируются при валидации.

backend/app/schemas/upload.py Pydantic-схема ответа на загрузку изображения (например, фото мастера, точки или главного экрана) – возвращает URL сохранённого файла.

```
class ImageUploadResponse(BaseModel)    # ответ API с URL
    ↪ загрузенного изображения
```

6 Backend: репозитории (repositories/)

Один класс-репозиторий на одну сущность БД. Единственное место во всём проекте, где пишутся SQLAlchemy-запросы (`select`, `insert` и т.д.) – сервисы (раздел 7) никогда не составляют SQL напрямую. Общей базовой абстракции репозитория или generic CRUD-класса в проекте осознанно нет – при относительно небольшом числе репозиториях (15 штук) наследование добавило бы косвенности больше, чем сэкономило бы кода; переиспользуемая логика (пагинация, экранирование `LIKE`) вынесена свободными функциями в `_query_utils.py` – он и описан первым.

backend/app/repositories/_query_utils.py Общий модуль с двумя SQL-утилитами, переиспользуемыми почти во всех репозиториях с поиском/списками: безопасное экранирование текста для `ILIKE`-поиска и универсальная пагинация (страница + общее число) поверх готового запроса.

```
LIKE_ESCAPE_CHAR: str    # константа "\\\"
escape_like(value: str) -> str
paginated(db: Session, stmt: Select[tuple[T]], *, page: int,
    ↪ page_size: int) -> tuple[list[T], int]
```

Особенности: без `escape_like` поиск по строке с `%` или `_` работал бы некорректно и мог провоцировать дорогие полные сканы таблицы – использовать строго в паре с `.ilike(f"%{escape_like(s)}%", escape=LIKE_ESCAPE_CHAR)`.

backend/app/repositories/appointment_repository.py

Разобран целиком в разделе 3.3 (шаг 4) – запись/чтение записей на приём, включая поиск пересечений и выборку записей для рассылки напоминаний. Полный список методов – в самом файле (раздел 3.3 приводит лишь `create()`).

backend/app/repositories/user_repository.py

Центральный репозиторий пользователей – поиск по email/телефону/VK ID, список для админ-панели с фильтрами по роли и точке сети, создание (обычное и через VK OAuth), смена пароля/роли/точки/блокировки, мягкое удаление.

Разобран частично в разделе 10 (аутентификация); полный список методов – в самом файле.

backend/app/repositories/master_repository.py Центральный репозиторий профилей мастеров – каталог мастеров с фильтрами и сортировкой, привязка мастера к точке сети, список услуг мастера с индивидуальными ценами, мягкое удаление и деактивация профиля.

```
get_by_id(master_id, *, include_deleted=False) -> Master | None
get_by_user_id(user_id) -> Master | None
list_paginated(*, page, page_size, specialization=None,
    ↪ service_id=None,
        salon_id=None, is_active=True, sort_by="name",
        ↪ sort_order="asc") -> tuple[list[Master], int]
get_with_services(master_id) -> Master | None
create(user_id, salon_id) -> Master
update(master, data: MasterUpdate) -> Master
deactivate(master) -> None
reactivate(master) -> Master
soft_delete(master) -> None
add_service(master_id, service_id, price_override=None) ->
    ↪ MasterService
remove_service(master_id, service_id) -> bool
get_master_service(master_id, service_id) -> MasterService | None
```

Особенности: при сортировке по цене с выбранной услугой сортировка идёт по реальной итоговой цене (`price_override`, если задан, иначе базовая цена $\times$ `coefficient`); `reactivate()` восстанавливает старый профиль мастера (специализацию, коэффициент, услуги, историю отзывов) вместо создания нового при повторном назначении роли.

backend/app/repositories/salon_repository.py Репозиторий точек сети (салонов) – адреса, часы работы, slug для URL; здесь же создание/редактирование точки, включая метод полной замены данных для сценария первичной настройки.

```
get_by_id(salon_id) -> Salon | None
get_by_slug(slug) -> Salon | None
get_first() -> Salon | None
list_all(*, is_active=None) -> list[Salon]
```

```
create(data: SalonCreate, slug: str) -> Salon
update(salon, data: SalonUpdate) -> Salon
replace_details(salon, data: SalonCreate, slug: str) -> Salon
```

Особенности: `list_all` намеренно без пагинации – у сети физически десятки точек максимум; `replace_details` нужен визарду `/api/setup`, чтобы заполнить строку-заглушку из миграции реальными данными вместо создания второго салона.

backend/app/repositories/service_repository.py Репозиторий каталога услуг – базовая цена, длительность, поиск по названию, фильтр по цене и активности, пагинация и сортировка. Услуги удаляются мягко, чтобы не сломать историю уже сделанных записей.

```
get_by_id(service_id, *, include_deleted=False) -> Service | None
list_paginated(*, page, page_size, search=None, min_price=None,
    ↪ max_price=None,
        is_active=None, sort_by="name", sort_order="asc")
    ↪ -> tuple[list[Service], int]
create(data: ServiceCreate) -> Service
update(service, data: ServiceUpdate) -> Service
soft_delete(service) -> None
```

Особенности: `soft_delete` не удаляет строку физически, а проставляет `deleted_at` и `is_active=False` – услуга остаётся доступной для уже существующих записей клиентов (`onDelete=RESTRICT`).

backend/app/repositories/schedule_repository.py Репозиторий недельного расписания мастера – какие дни недели мастер работает и в какие часы. У одного мастера может быть до семи записей расписания, по одной на каждый день недели.

```
get_by_id(schedule_id) -> Schedule | None
get_by_master(master_id) -> list[Schedule]
get_by_master_and_day(master_id, day_of_week) -> Schedule | None
create(master_id, data: ScheduleCreate) -> Schedule
update(schedule, data: ScheduleUpdate) -> Schedule
```

backend/app/repositories/review_repository.py Репозиторий отзывов клиентов на мастеров – чтение с фильтрами (мастер, услуга, минимальный рейтинг, статус публикации) и пагинацией, создание, публикация/скрытие модератором, удаление, средний рейтинг мастера.

```
get_by_id(review_id) -> Review | None
get_by_appointment(appointment_id) -> Review | None
list_paginated(*, page, page_size, master_id=None,
    ↪ service_id=None, min_rating=None,
        is_published=True, salon_id=None,
        ↪ sort_order="desc") -> tuple[list[Review],
        ↪ int]
rating_summary(master_id) -> tuple[float | None, int]
create(*, appointment_id, client_id, master_id, service_id,
    ↪ rating, comment) -> Review
set_published(review, is_published) -> Review
delete(review) -> None
```

Особенности: `is_published=None` переключает в режим модерации (видно всё, включая скрытые); у модели `Review` нет собственного `salon_id`, поэтому скоуп по точке для администратора делается через `join` на мастера отзыва.

backend/app/repositories/session_repository.py Репозиторий пользовательских сессий – серверной части `refresh`-токенов, позволяющих продлевать вход без повторного ввода пароля.

```
create(user_id, token_hash, expires_at) -> Session
get_valid_by_hash(token_hash) -> Session | None
delete(session) -> None
delete_by_hash(token_hash) -> None
delete_all_for_user(user_id) -> None
```

Особенности: `delete_all_for_user` вызывается вместе с повышением версии токена (`bump_token_version`) на событиях безопасности – так закрывается полный логат со всех устройств (раздел 10).

backend/app/repositories/password_reset_token_repository.py Хранит и проверяет одноразовые токены сброса пароля.

```
get_valid_by_hash(token_hash) -> PasswordResetToken | None
create(user_id, token_hash, expires_at) -> PasswordResetToken
count_created_since(user_id, since) -> int
invalidate_all_for_user(user_id) -> None
mark_used(token) -> None
```

Особенности: валидность = не использован и не просрочен; новый запрос сброса гасит все ранее выданные неиспользованные токены пользователя.

backend/app/repositories/email_verification_token_repository.py Репозиторий токенов подтверждения email – по структуре и назначению почти зеркален `password_reset_token_repository.py`.

```
get_valid_by_hash(token_hash) -> EmailVerificationToken | None
create(user_id, token_hash, expires_at) -> EmailVerificationToken
count_created_since(user_id, since) -> int
invalidate_all_for_user(user_id) -> None
mark_used(token) -> None
```

backend/app/repositories/login_attempt_repository.py

Журналирует каждую попытку входа в систему (успешную и неуспешную). Записи используются для аудита безопасности и для временной блокировки аккаунта после серии неудачных попыток входа подряд.

```
create(*, email_attempted, user_id, ip_address, success) ->
↳ LoginAttempt
count_recent_failed(email, since) -> int
```

Особенности: `count_recent_failed` покрыт индексом `ix_login_attempts_email_created`, чтобы подсчёт неудачных попыток за окно времени оставался быстрым при большом журнале.

backend/app/repositories/site_settings_repository.py

Репозиторий редактируемого контента сайта, который хранится не как набор колонок, а как один JSON-объект в единственной строке таблицы.

```
get() -> SiteSettings | None
create() -> SiteSettings
update(settings, content: dict) -> SiteSettings
```

Особенности: таблица рассчитана ровно на одну строку – `get()` берёт первую попавшуюся запись без фильтра по `id`.

backend/app/repositories/stats_repository.py Агрегирующие запросы для счётчиков и графиков главной страницы админ-панели – пользователи по ролям, активные мастера и услуги, выручка и визиты с начала месяца, график регистраций за 30 дней.

```
count_users_by_role() -> dict[UserRole, int]
count_active_masters(salon_id=None) -> int
count_active_services() -> int
appointments_and_revenue_since(since, salon_id=None) ->
    ↳ tuple[int, Decimal]
daily_registrations(since) -> list[tuple[datetime, int]]
```

Особенности: total_users/registrations_last_30_days/total_services намеренно остаются сетевыми всегда (без salon_id), а мастера/выручка скоупятся по точке.

backend/app/repositories/report_repository.py Агрегирующие SQL-запросы для экрана аналитики администратора – выручка, число визитов, процент повторных клиентов, разбивка по дням/услугам/мастерам за период, только по завершённым записям (status = done).

```
get_summary(date_from, date_to, salon_id=None) -> tuple[int,
    ↳ Decimal, Decimal]
get_repeat_clients_pct(date_from, date_to, salon_id=None) ->
    ↳ float
get_revenue_by_day(date_from, date_to, salon_id=None) ->
    ↳ list[tuple]
get_appointments_by_service(date_from, date_to, salon_id=None) ->
    ↳ list[tuple]
get_masters_breakdown(date_from, date_to, salon_id=None) ->
    ↳ list[tuple]
```

Особенности: границы периода и группировка по дням явно приводятся к UTC, чтобы не зависеть от таймзоны сессии PostgreSQL; «повторный клиент» считается по точке, в которую клиент вернулся, а не по точке первого визита.

7 Backend: сервисы (services/)

Вся бизнес-логика проекта живёт здесь. Сервис вызывает один или несколько репозиториев и ничего не знает про HTTP – ни Request, ни коды статусов сам не выбирает без причины (HTTPException внутри сервиса – это осознанный ответ бизнес-правила, а не техническая деталь транспорта). AppointmentService и AuthService (самые связанные сервисы проекта) разобраны подробно в разделах 3.3 и 10 соответственно.

backend/app/services/_errors.py Общая утилита сервисного слоя для превращения ошибки целостности БД (IntegrityError) в понятное имя нарушенного ограничения – используется во всех сервисах, где перед записью в БД делается предварительная проверка вида «email уже занят» (тот же приём, что и в шаге 6 раздела 3.3).

```
constraint_name(exc: IntegrityError) -> str | None
```

Особенности: предварительные проверки в сервисах – это check-then-insert с окном для гонки (два одновременных запроса могут оба пройти проверку); настоящую целостность гарантируют ограничения PostgreSQL, а проигравший гонку запрос обязан получить корректный 409 через перехват IntegrityError, а не случайный 500.

backend/app/services/appointment_service.py Самый связанный сервис проекта – разобран целиком в разделе 3.3 (создание записи) и упомянут в разделе 9 (константа ALLOWED_TRANSITIONS, машина состояний). Кроме create(), в файле – отмена, смена статуса, перенос записи и расчёт свободных слотов; все методы стоит прочитать вместе, они разделяют одну и ту же проверку прав доступа (владелец записи / её мастер / администратор своей точки / владелец сети).

backend/app/services/admin_service.py Самый крупный сервис приложения – бизнес-логика админ-панели: управление пользователями (создание, редактирование, смена роли, привязка к точке сети, блокировка, мягкое удаление), удаление услуг, подсчёт статистики дашборда. Здесь принимаются решения о правах – кто кому какую роль может назначить.

```

_ensure_can_assign_role(role: UserRole, requesting_user: User) ->
    ↪ None
list_users(*, page, page_size, role=None, search=None,
    ↪ salon_id=None,
        sort_by="created_at", sort_order="desc") ->
    ↪ PageResponse[UserResponse]
create_user(data: AdminUserCreate, requesting_user) ->
    ↪ UserResponse
update_user(user_id, data: AdminUserUpdate) -> UserResponse
change_role(user_id, role, requesting_user) -> UserResponse
create_master_profile(user_id, salon_id, requesting_user) ->
    ↪ MasterResponse
assign_salon(user_id, salon_id) -> UserResponse
set_blocked(user_id, is_blocked, requesting_admin_id) ->
    ↪ UserResponse
delete_user(user_id) -> None
delete_service(service_id) -> None
get_stats(salon_id=None) -> AdminStatsResponse

```

Особенности: назначить роль `admin/owner` может только `owner` – иначе администратор точки мог бы выдать себе роль владельца и захватить всю сеть; блокировка и мягкое удаление сразу отзывают все `refresh`-сессии и повышают `token_version`; создать пользователя сразу с ролью `admin` через `create_user` нельзя – сначала нужно назначить точку, затем сменить роль (ограничение БД `ck_users_admin_requires_salon`).

backend/app/services/auth_service.py Разобран подробно в разделе 10 – вход/регистрация, работа с JWT-токенами, восстановление пароля, подтверждение email, зависимости `get_current_user/require_role`.

backend/app/services/vk_oauth_service.py Разобран подробно в разделе 10 – вход через VK ID (OAuth 2.1 + PKCE).

backend/app/services/_salon_scope.py Разобран подробно в разделе 10 – функция `resolve_salon_scope`, превращающая параметр запроса `salon_id` в реально применяемый фильтр по точке сети в зависимости от роли вызывающего.

backend/app/services/master_service.py Бизнес-логика вокруг профиля мастера – публичный и приватный варианты профиля, список услуг мастера с рассчитанной итоговой ценой, управление недельным расписанием с проверкой, что оно не выходит за часы работы точки.

```
_final_price(price_override, base_price, coefficient) -> Decimal
list_paginated(*, page, page_size, specialization=None,
    ↪ service_id=None, salon_id=None,
        is_active=True, sort_by="name", sort_order="asc")
    ↪ -> PageResponse[MasterBriefResponse]
get_by_id(master_id) -> MasterResponse
get_public_by_id(master_id) -> MasterPublicResponse
update(master_id, data: MasterUpdate) -> MasterResponse
get_services(master_id) -> list[MasterServiceResponse]
add_service(master_id, service_id, price_override=None) ->
    ↪ MasterServiceResponse
remove_service(master_id, service_id) -> None
get_schedule(master_id) -> list[ScheduleResponse]
set_schedule(master_id, data: ScheduleCreate) -> ScheduleResponse
update_schedule(master_id, day_of_week, data: ScheduleUpdate) ->
    ↪ ScheduleResponse
```

Особенности: расписание мастера не может выходить за часы работы его точки (та же проверка задублирована как бэкстоп в `AppointmentService`, раздел 3.3); `get_public_by_id` скрывает деактивированных мастеров и не отдаёт контакты пользователя; `set_schedule` – upsert (повторная установка на тот же день перезаписывает запись).

backend/app/services/salon_service.py Бизнес-логика точек сети – список, получение по id, создание новой точки и её редактирование, плюс отдельный метод для первичной настройки инсталляции, который заполняет уже существующую точку-заглушку вместо создания второй.

```
list_all(*, is_active=None) -> list[SalonResponse]
get_by_id(salon_id) -> SalonResponse
create(data: SalonCreate) -> SalonResponse
ensure_primary(data: SalonCreate) -> SalonResponse
update(salon_id, data: SalonUpdate) -> SalonResponse
```

Особенности: `ensure_primary` заполняет данными строку-заглушку («Салон №1», созданную миграцией 0013), а не создаёт вторую точку.

backend/app/services/service_service.py Бизнес-логика каталога услуг – тонкая обёртка над репозиторием без дополнительных правил.

```
list_paginated(*, page, page_size, search=None, min_price=None,
    ↪ max_price=None,
    is_active=None, sort_by="name", sort_order="asc")
    ↪ -> PageResponse[ServiceResponse]
get_by_id(service_id) -> ServiceResponse
create(data: ServiceCreate) -> ServiceResponse
update(service_id, data: ServiceUpdate) -> ServiceResponse
```

Особенности: в отличие от соседних сервисов (SalonService, MasterService) здесь нет дополнительной бизнес-валидации – вся логика сводится к поиску записи и делегированию репозиторию, право доступа (только владелец сети) проверяется целиком на уровне роута.

backend/app/services/review_service.py Бизнес-правила вокруг отзывов клиентов – оставить отзыв можно только на свою и только на завершённую запись, и только один раз; публичный и админский список отзывов; модерация и удаление с разграничением прав по точке сети.

```
create(client_id, data: ReviewCreate) -> ReviewResponse
list_for_master(master_id, *, page, page_size, min_rating=None)
    ↪ -> PageResponse[ReviewResponse]
list_all_admin(*, page, page_size, master_id=None,
    ↪ is_published=None,
    min_rating=None, salon_id=None) ->
    ↪ PageResponse[ReviewResponse]
moderate(review_id, is_published, requesting_user) ->
    ↪ ReviewResponse
delete(review_id, requesting_user) -> None
```

Особенности: владелец сети модерирует и удаляет отзывы любой точки, администратор – только своей точки (скоуп через мастера отзыва); обычный клиент может удалить только собственный отзыв.

backend/app/services/reminder_service.py Рассылает клиентам напоминания о записи за 24 часа и за 2 часа до визита. Сервис не привязан к расписанию каждой конкретной записи – периодически опрашивается фоновым процессом (app/scheduler.py, раздел 3.2).

```
send_due_reminders() -> None
```

Особенности: одна проблемная запись (например, отсутствующая связь) не останавливает рассылку остальным клиентам в этом же проходе – ошибка ловится точно на каждой записи и пишется в лог, а не поднимается наружу; `send_email` сама никогда не бросает исключение при сбое SMTP.

backend/app/services/report_service.py Собирает данные аналитического отчёта админ-панели в единый ответ API, а также умеет отдать тот же отчёт файлом Excel для скачивания.

```
_write_table_header(ws, row, columns) -> None  
get_report(date_from, date_to, salon_id=None) -> ReportResponse  
export_excel(date_from, date_to, salon_id=None) -> bytes
```

Особенности: денежные суммы остаются типом `Decimal` вплоть до сериализации; Excel-файл строится из 4 листов (Сводка, Выручка по дням, По услугам, Мастера) через `openpyxl`.

backend/app/services/setup_service.py Сервис первичной настройки после развёртывания – создаёт первого владельца сети, первую точку и, опционально, базовый контент сайта одним запросом от визарда на фронте. После первого успешного вызова эндпоинт навсегда «закрывается».

```
_SETUP_LOCK_KEY = 727271 # константа модуля  
is_completed() -> bool  
requires_token() -> bool  
complete(data: SetupRequest) -> TokenResponse
```

Особенности: `complete()` берёт `pg_advisory_xact_lock`, чтобы два одновременных запроса не смогли оба пройти проверку «ещё не настроено» и оба создать «первого» владельца; после завершения настройки повторные вызовы отдадут 404, а не 409, чтобы не подтверждать анонимному запросу сам факт настройки; код настройки сравнивается через `secrets.compare_digest` (защита от атаки по времени ответа).

backend/app/services/site_settings_service.py Простой сервис редактируемого контента сайта – получить текущий контент (создав запись по умолчанию, если её ещё нет) и обновить его.

```
get() -> SiteContent  
update(data: SiteContent) -> SiteContent
```

8 Backend: роуты (routes/)

FastAPI-роутеры – самый тонкий слой (раздел 3.3): проверяют роль через Depends, вызывают один метод сервиса, отдают результат. У каждого метода помечено, какая зависимость роли (раздел 10) его защищает. routes/appointments.py и routes/auth.py разобраны подробно в разделах 3.3 и 10.

backend/app/routes/admin.py Роутер /api/admin – административная часть API: статистика и аналитические отчёты (включая выгрузку в Excel), полное управление пользователями (список, создание, редактирование, смена роли, привязка к точке, блокировка, создание профиля мастера, мягкое удаление), редактирование/удаление услуг, загрузка изображений и обновление фото мастера.

```
GET      /api/admin/stats                -> AdminStatsResponse
  ↳ [admin]
GET      /api/admin/reports            -> ReportResponse
  ↳ [admin]
GET      /api/admin/reports/export      -> файл .xlsx
  ↳ [admin]
GET      /api/admin/users              ->
  ↳ PageResponse[UserResponse] [admin]
POST     /api/admin/users              -> UserResponse (201)
  ↳ [admin]
PATCH   /api/admin/users/{user_id}    -> UserResponse
  ↳ [admin]
PATCH   /api/admin/users/{user_id}/role -> UserResponse
  ↳ [admin]
PATCH   /api/admin/users/{user_id}/salon -> UserResponse
  ↳ [owner]
PATCH   /api/admin/users/{user_id}/block -> UserResponse
  ↳ [admin]
POST     /api/admin/users/{user_id}/master -> MasterResponse
  ↳ (201) [admin]
DELETE   /api/admin/users/{user_id}    -> 204
  ↳ [admin]
PATCH   /api/admin/services/{service_id} -> ServiceResponse
  ↳ [owner]
```

```

DELETE /api/admin/services/{service_id}    -> 204
  ↳ [owner]
POST    /api/admin/uploads/image            -> ImageUploadResponse
  ↳ (201) [admin]
PATCH  /api/admin/masters/{master_id}/photo -> MasterResponse
  ↳ [admin]

```

Особенности: право доступа разное по эндпоинтам одного файла – большинству операций достаточно роли `admin` (в коде – `get_current_admin`, которая пропускает и `owner`, раздел 1), но привязка пользователя к точке сети и любые изменения каталога услуг требуют именно `get_current_owner`; фактический скоуп по точке (своя точка для админа, любая – для владельца) вычисляется общей функцией `resolve_salon_scope` (раздел 10).

backend/app/routes/masters.py Роутер `/api/masters` – публичный каталог мастеров с фильтрами, личный кабинет мастера, публичный профиль конкретного мастера, управление профилем/услугами/расписанием и получение свободных слотов для записи.

```

GET      /api/masters                                ->
  ↳ PageResponse[MasterBriefResponse]  [публичный]
GET      /api/masters/me                            -> MasterResponse
  ↳ [master]
GET      /api/masters/{master_id}                   ->
  ↳ MasterPublicResponse                [публичный]
PATCH   /api/masters/{master_id}                   -> MasterResponse
  ↳ [master]
GET      /api/masters/{master_id}/services          ->
  ↳ list[MasterServiceResponse]         [публичный]
POST     /api/masters/{master_id}/services          ->
  ↳ MasterServiceResponse (201)         [admin]
DELETE   /api/masters/{master_id}/services/{id}     -> 204
  ↳ [admin]
GET      /api/masters/{master_id}/schedule          ->
  ↳ list[ScheduleResponse]              [публичный]
POST     /api/masters/{master_id}/schedule          ->
  ↳ ScheduleResponse                    [master]
PATCH   /api/masters/{master_id}/schedule/{day}    ->
  ↳ ScheduleResponse                    [master]
GET      /api/masters/{master_id}/slots             ->
  ↳ SlotListResponse                    [публичный]

```

Особенности: внутренняя проверка `_ensure_can_manage_master` – трёхуровневая (мастер управляет только собой, администратор – только мастерами своей точки, владелец – всеми), но перевод мастера в другую точку сети (`salon_id`) разрешён только владельцу даже на эндпоинте, в остальном доступном мастеру/администратору.

backend/app/routes/salons.py Роутер `/api/salons` – публичный список точек сети без пагинации, детали одной точки, создание новой точки и её редактирование.

```
GET    /api/salons                -> list[SalonResponse]
  ↳ [опционально авторизован]
GET    /api/salons/{salon_id}  -> SalonResponse
  ↳ [публичный]
POST   /api/salons                -> SalonResponse (201)   [owner]
PATCH /api/salons/{salon_id}  -> SalonResponse        [admin]
```

Особенности: администратор может редактировать поля своей точки, но не может её открыть/закрыть (`is_active`) – это решение уровня всей сети, зарезервированное для владельца; неавторизованным и клиентам всегда показываются только активные точки.

backend/app/routes/services.py Роутер `/api/services` – публичный каталог услуг с поиском/фильтром по цене/сортировкой, детали одной услуги, создание и редактирование услуги.

```
GET    /api/services              -> PageResponse[ServiceResponse]
  ↳ [опционально авторизован]
GET    /api/services/{service_id} -> ServiceResponse
  ↳ [публичный]
POST   /api/services              -> ServiceResponse (201)
  ↳ [owner]
PATCH /api/services/{service_id} -> ServiceResponse
  ↳ [owner]
```

Особенности: каталог услуг – общий актив сети, поэтому создавать и редактировать его может только владелец; локальная вариативность цены по точке остаётся доступной администратору через `price_override` на конкретном мастере (`masters.py`).

backend/app/routes/reviews.py Роутер `/api/reviews` – оставить отзыв на завершённую запись, публичный список отзывов конкретного мастера, полный список для модерации администратором, публикация/скрытие и удаление отзыва.

```
POST    /api/reviews                                -> ReviewResponse (201)
  ↳ [client, лимит запросов]
GET     /api/reviews/master/{master_id}            ->
  ↳ PageResponse[ReviewResponse] [публичный]
GET     /api/reviews                                ->
  ↳ PageResponse[ReviewResponse] [admin]
PATCH  /api/reviews/{review_id}/publish           -> ReviewResponse
  ↳ [admin]
DELETE  /api/reviews/{review_id}                   -> 204
  ↳ [client]
```

Особенности: создание отзыва ограничено лимитом запросов (раздел 10) – защита от спама отзывами; удаление формально висит на `get_current_client`, но фактическое право (свой отзыв vs. любой отзыв своей точки vs. любой отзыв сети) разруливает `ReviewService.delete`.

backend/app/routes/setup.py Роутер `/api/setup` – статус первичной настройки (нужен ли фронтенду показать визард) и сам визард, который одним запросом создаёт владельца сети, первую точку и сразу логинит нового владельца.

```
GET    /api/setup/status  -> SetupStatusResponse [публичный]
POST   /api/setup         -> TokenResponse (201)  [публичный]
```

Особенности: эндпоинт неизбежно публичный (до первого вызова в системе нет управляющего аккаунта, которым можно было бы его защитить) и сразу выставляет `auth-` и `refresh-cookie`, логиня нового владельца без отдельного запроса на вход.

backend/app/routes/site_settings.py Роутер `/api/settings` – публичное чтение редактируемого контента сайта (шапка, главная, футер) и его обновление владельцем сети.

```
GET    /api/settings  -> SiteContent [публичный]
PATCH /api/settings  -> SiteContent [owner]
```

9 Backend: утилиты (utils/)

Небольшие независимые модули без состояния – каждый решает одну узкую задачу и используется из нескольких мест сразу.

backend/app/utils/rate_limit.py Разобран подробно в разделе 10 – общий объект `limiter` (`slowapi`), ограничивающий частоту запросов по IP.

backend/app/utils/client_ip.py Утилита определения настоящего IP-адреса клиента для лимитов запросов и журналирования попыток входа. Так как приложение всегда работает за обратным прокси (Caddy → nginx, раздел 19), `request.client.host` показал бы IP контейнера nginx, а не реального посетителя.

```
client_ip(request: Request) -> str
```

Особенности: функция берёт первый IP в цепочке X-Forwarded-For, и это безопасно от подделки именно потому, что у Caddy не настроены доверенные прокси иначе, чем свои же контейнеры – он сам переписывает заголовок реальным увиденным IP.

backend/app/utils/content_filter.py Фильтр нежелательного контента для пользовательского текста (в первую очередь – комментариев к отзывам): проверка на нецензурную лексику и на попытки оставить ссылку/домен в тексте.

```
_clean_word(word: str) -> str  
contains_profanity(text: str | None) -> bool  
contains_link(text: str | None) -> bool
```

Особенности: список бранных корней короткий и по префиксам (не общие слоги, чтобы не ловить обычные слова), а нормализация схлопывает 3+ повторяющихся буквы и переводит латинские двойники в кириллицу – ловит обход фильтра, не портя слова с обычными двойными буквами.

backend/app/utils/email.py Тонкая обёртка над отправкой писем по SMTP (по умолчанию – локальный Mailpit для разработки, раздел 20) – используется для писем подтверждения регистрации, сброса пароля и напоминаний о записи.

```
send_email(*, to: str, subject: str, text_body: str, html_body:
    ↳ str | None = None) -> None
```

Особенности: функция никогда не поднимает исключение при сбое SMTP (ошибка только логируется) – это намеренно, чтобы, например, сценарий восстановления пароля не раскрывал через различие в ответе, существует ли указанный email в системе (раздел 10).

backend/app/utils/slug.py Генератор человекочитаемых URL-идентификаторов (slug) для точек сети – транслитерация кириллического названия в латиницу и подбор уникального варианта при совпадении с уже существующим slug.

```
slugify(text: str) -> str
generate_unique_slug(name: str, *, exists: Callable[[str], bool],
    ↳ fallback: str = "salon") -> str
```

Особенности: проверка уникальности передаётся как функция-предикат `exists`, а не завязана на конкретный репозиторий – это осознанно, чтобы `generate_unique_slug` можно было протестировать без обращения к БД.

backend/app/utils/uploads.py Сохраняет загруженное через админку изображение (фото мастера, hero-картинка настроек сайта) на диск и возвращает публичный URL, с проверкой типа файла и ограничением размера.

```
save_uploaded_image(file: UploadFile) -> str
```

Особенности: имя файла на диске всегда генерируется сервером (uuid4 + расширение из белого списка content-type), а не берётся из пользовательского ввода – исключает path traversal и загрузку произвольного не-картиночного файла.

10 Аутентификация и безопасность

Механизм авторизации – один из самых плотно связанных участков кода проекта (`User` и `Session` – два самых «подключённых» класса во всём графе кода проекта). Этот раздел разбирает его целиком: как устроен вход, откуда роут узнаёт роль вызывающего, как отзываются токены и как проект защищается от подбора пароля.

10.1 Два токена вместо одного

Вместо одного долгоживущего токена проект использует пару: короткоживущий JWT **access-токен** (15 минут, `settings.access_token_expire_minutes`) и долгоживущий opaque **refresh-токен** (30 дней, хранится как строка в таблице `sessions`, раздел 11). Оба живут в отдельных `httpOnly-cookie`, а не в `localStorage`/JS-переменной – так украсть их через XSS-уязвимость на фронтенде существенно сложнее (JavaScript их вообще не видит).

```
ACCESS_TOKEN_COOKIE = "access_token"      # path="/api", уходит с
↳ каждым запросом к API
REFRESH_TOKEN_COOKIE = "refresh_token"    # path="/api/auth",
↳ уходит только к эндпоинтам /auth
```

Access-токен – обычный JWT (подписан HS256, `backend/app/services/auth_service.py`):

```
def _create_access_token(user: User) -> str:
    expire = datetime.now(timezone.utc) +
    ↳ timedelta(minutes=settings.access_token_expire_minutes)
    payload = {"sub": str(user.id), "ver": user.token_version,
    ↳ "exp": expire}
    return jwt.encode(payload, settings.secret_key,
    ↳ algorithm=settings.algorithm)
```

Поле `"ver"` – снимок `user.token_version` на момент выдачи токена. Это и есть механизм отзыва: JWT сам по себе никак нельзя «отменить» до истечения срока действия, но `get_current_user` при каждой проверке сверяет `ver` из токена с текущим значением в БД:

```

def get_current_user(request, credentials=Depends(...),
↳ db=Depends(get_db)) -> User:
    token = _token_from_request(request, credentials)
    ...
    user_id, token_version = _decode_token(token)
    user = UserRepository(db).get_by_id(user_id)
    if user.token_version != token_version:
        raise HTTPException(status_code=401, detail="Токен
↳ отозван, войдите заново")
    if user.is_blocked:
        raise HTTPException(status_code=403, detail="Аккаунт
↳ заблокирован...")
    return user

```

`token_version` инкрементируется (`UserRepository.bump_token_version`) при `logout`, смене/сбросе пароля и блокировке – любой уже выданный access-токен с прежним `ver` мгновенно перестаёт проходить эту проверку, без хранения отдельного «чёрного списка» токенов.

Refresh-токен работает иначе – он не JWT, а случайная строка (`secrets.token_urlsafe(32)`), хеш которой (SHA-256) хранится в таблице `sessions`. Каждый вызов `POST /api/auth/refresh` *ротуирует* токен – старая сессия удаляется, выдаётся новая:

```

def refresh_session(self, raw_refresh_token: str) ->
↳ tuple[TokenResponse, str]:
    session = self.session_repo.get_valid_by_hash(_hash_token(raw_
↳ w_refresh_token))
    if not session:
        raise HTTPException(status_code=401, detail="Сессия
↳ недействительна...")
    user = self.user_repo.get_by_id(session.user_id)
    self.session_repo.delete(session)
    ...
    new_raw_token = create_session(self.db, user.id)
    return build_token_response(user), new_raw_token

```

Смысл ротации: если старый refresh-токен всё же попытаются использовать повторно (например, он был украден и злоумышленник, и легитимный клиент пробуют обновиться одновременно), второй из двух получит 401 вместо тихой утечки бессрочного доступа – сессии уже нет, она удалена первым же успешным запросом.

10.2 Роли и `require_role`

Проверка роли – отдельная функция-фабрика, а не набор дублирующихся проверок в каждом роуте:

```
def require_role(*roles: UserRole):
    def dependency(current_user: User =
        ↳ Depends(get_current_user)) -> User:
        if current_user.role not in roles:
            raise HTTPException(status_code=403,
                ↳ detail="Недостаточно прав")
        return current_user
    return dependency

get_current_client = require_role(UserRole.client,
    ↳ UserRole.admin, UserRole.owner)
get_current_master = require_role(UserRole.master,
    ↳ UserRole.admin, UserRole.owner)
get_current_admin = require_role(UserRole.admin, UserRole.owner)
    ↳ # owner неявно включён
get_current_owner = require_role(UserRole.owner)
```

Именно эти четыре готовые зависимости и стоят в объявлении `Depends(...)` у каждого роута из разделов 8 (там же – пометка `[admin]/[owner]/...` у каждого эндпоинта, которая буквально означает «за какой из этих четырёх функций он стоит»).

10.3 Разрешение области видимости по точке сети

Роль решает, *что* можно делать; отдельная функция решает, *в какой точке сети*. `backend/app/services/_salon_scope.py` – всего 20 строк, но вызывается из большинства «списочных» эндпоинтов админки (раздел 8):

```
def resolve_salon_scope(current_user: User, salon_id_param: UUID
    ↳ | None) -> UUID | None:
    if current_user.role == UserRole.owner:
        return salon_id_param
    if salon_id_param is not None and salon_id_param !=
        ↳ current_user.salon_id:
```

```

        raise HTTPException(status_code=403, detail="Доступен
        ↪ только собственный салон")
    return current_user.salon_id

```

Правило: для `owner` отсутствие параметра означает «вся сеть», а явно переданный `salon_id` – «покажи мне эту одну точку». Для `admin` результат всегда его собственная точка – запрос с чужим `salon_id` в параметре получает не тихую подмену на правильное значение, а явный 403. Эта функция – обычная функция, не `Depends`: вызывается уже *внутри* эндпоинтов, защищённых ролью `get_current_admin` (то есть после того, как FastAPI уже убедился, что вызывающий – как минимум администратор какой-то точки).

10.4 Вход через VK ID (OAuth 2.1 + PKCE)

`backend/app/services/vk_oauth_service.py` и руты `/api/auth/vk/*` реализуют классический `redirect-флоу` (не SPA-попап): фронтенд просто делает полный переход браузера на `GET /api/auth/vk/login`, а не `axios`-запрос – потому что серверу нужно выставить `cookie` и сделать редирект на `id.vk.com`, что может только сам браузер.

```

@router.get("/vk/login")
def vk_login():
    state = secrets.token_urlsafe(24)
    verifier, challenge = generate_pkce()
    redirect = RedirectResponse(build_authorize_url(state,
    ↪ challenge))
    for key, value in ((STATE_COOKIE, state), (VERIFIER_COOKIE,
    ↪ verifier)):
        redirect.set_cookie(key=key, value=value,
        ↪ max_age=COOKIE_MAX_AGE,
                                path=COOKIE_PATH, httponly=True,
                                ↪ samesite="lax")
    return redirect

```

`generate_pkce()` создаёт пару (`code_verifier`, `code_challenge`) – PKCE (Proof Key for Code Exchange) защищает от перехвата кода авторизации по пути, даже без секрета клиента. `state` кладётся в `cookie` и на обратном пути (`GET /api/auth/vk/callback`) сверяется с тем, что VK

вернул параметром запроса – это и есть защита от CSRF на этом конкретном эндпоинте:

```
if not (code and device_id and state and cookie_state and
    ↪ verifier) or state != cookie_state:
    return fail("vk_auth_failed")
```

Дальше `VkOAuthService.handle_callback` обменивает код на токен VK, получает профиль пользователя и решает, что делать: если `vk_user_id` уже привязан к аккаунту – логинит; если найден пользователь с тем же email, зарегистрированный по паролю, – привязывает VK к этому аккаунту (email от VK считается доказанным владением, письмо подтверждения не отправляется повторно); иначе создаёт нового пользователя без пароля (`password_hash=None` – такой аккаунт никогда не пройдёт обычный вход по email/паролю).

10.5 Защита от подбора пароля и лимиты запросов

Два независимых механизма, оба работают в памяти процесса backend, без Redis – проект и так рассчитан ровно на один работающий экземпляр контейнера backend (раздел 3.2), поэтому отдельное общее хранилище не добавляет пользы, но добавляет эксплуатационную сложность.

Блокировка входа после серии неудач. Таблица `login_attempts` (раздел 11) журналирует *каждую* попытку входа, успешную и нет (требование к логированию попыток авторизации). `AuthService.login` перед проверкой пароля сначала считает недавние неудачи:

```
recent_failed =
    ↪ self.login_attempt_repo.count_recent_failed(email,
    ↪ window_start)
if recent_failed >= settings.login_max_failed_attempts:    # по
    ↪ умолчанию 5
    raise HTTPException(status_code=429, detail="Слишком много
        ↪ неудачных попыток...")
```

Тонкость, которую легко упустить при чтении кода поверхностно: `bcrypt` всё равно выполняется, даже если email вообще не зарегистрирован (сравнение идёт с заранее посчитанным `_DUMMY_HASH`). Без этого время ответа само по

себе выдавало бы, зарегистрирован ли адрес – запрос к несуществующему пользователю отвечал бы заметно быстрее, чем к существующему с неверным паролем.

Лимит частоты запросов (rate limiting). `backend/app/utils/rate_limit.py` – общий объект `limiter` (библиотека `slowapi`) по IP-адресу, а не по пользователю:

```
limiter = Limiter(key_func=client_ip)
```

Применяется декоратором прямо на эндпоинте: `@limiter.limit(settings.rate_limit_register)` на регистрации ("5/hour"), `rate_limit_booking` на создании записи ("20/minute"), `rate_limit_review` на отзывах ("10/hour"). Привязка по IP, а не по пользователю, неслучайна: `slowapi` проверяет лимит до того, как `FastAPI` успеваает прогнать остальные `Depends()` эндпоинта – на этом этапе `current_user` ещё физически недоступен. `client_ip()` (не `request.client.host`) берёт реальный IP из `X-Forwarded-For` – без этого за цепочкой прокси `Caddy` → `nginx` все запросы выглядели бы приходящими с одного и того же внутреннего IP контейнера.

10.6 Как cookie-авторизация выглядит со стороны фронтенда

Раздел 15 разбирает это подробно (файл `frontend/src/api/client.js`), но стоит связать с тем, что уже сказано выше: `axios` настроен с `withCredentials:true` (браузер сам прикладывает `httpOnly-cookie`, JS их не читает и не может прочитать); при ответе 401 интерцептор сам пробует `POST /api/auth/refresh` и повторяет исходный запрос – пользователь этого не замечает, пока не истечёт и `refresh-токен` тоже. Несколько одновременных 401 не запускают несколько параллельных `/refresh` – один общий `promise` на все (`single-flight`, раздел 15) – что согласуется с ротацией `refresh-токена` на `backend`: параллельный второй вызов `/refresh` с уже использованным токеном закономерно получил бы 401.

11 База данных и миграции

11.1 Alembic: схема БД – это история, а не снимок

Классы в `backend/app/models/` (раздел 4) описывают, *какой сейчас* должна быть схема БД – но сами по себе таблицы не создают. Создание и любое изменение схемы – отдельный файл-миграция в `backend/alembic/versions/`. Каждая миграция – обычный Python-файл с двумя функциями, `upgrade()` и `downgrade()`, и ссылкой на предыдущую миграцию (`down_revision`) – вместе они образуют цепочку версий схемы, которую можно накатить или откатить по шагам. Команда `alembic upgrade head` (запускается автоматически при старте контейнера `backend`, раздел 19) доводит базу данных до последней версии в этой цепочке. Отдельно от кода приложения миграции важны по практической причине: без них единственный способ применить изменение схемы на уже работающей базе с данными – вручную писать и выполнять `ALTER TABLE`.

11.2 Хронология: 15 миграций проекта

Таблица 4 – вся история схемы БД проекта, от первой миграции до последней. Читать её стоит по порядку сверху вниз – она одновременно служит кратким пересказом истории развития функционала: сначала базовая схема одного барбершопа, затем блокировка/JWT-отзыв/сессии/VK OAuth/подтверждение email по мере появления соответствующих требований, и в конце – переход на сеть из нескольких точек.

Таблица 4 – Все миграции backend/alembic/versions/

Миграция	Что меняет
0001_initial	Вся исходная схема одного барбершопа: <code>users</code> , <code>masters</code> , <code>services</code> , <code>master_services</code> , <code>schedules</code> , <code>appointments</code> (с ограничением <code>EXCLUDE USING gist</code> , раздел 11.3), <code>notifications</code> (позже удалена, см. 0005), <code>reviews</code> , <code>password_reset_tokens</code> , <code>login_attempts</code> , плюс SQL-функция <code>get_appointment_price()</code> (тоже позже удалена).
0002_site_settings	Добавляет singleton-таблицу <code>site_settings</code> – изначально только фото в шапке главной страницы.
0003_site_settings_content	Заменяет узкоспециальную колонку <code>hero_photo_url</code> на общую JSONB-колонку <code>content</code> , хранящую весь редактируемый текст сайта (см. <code>SiteContent</code> в разделе 5).
0004_masters_scheduled_created_at	Добавляет <code>created_at</code> таблицам <code>masters</code> и <code>schedules</code> – были единственными без этого столбца.
0005_drop_notifications	Удаляет таблицу <code>notifications</code> (три <code>enum</code> -типа) и функцию <code>get_appointment_price()</code> – ни один сервис или роут никогда не писал в эту таблицу, а цена всегда считалась в Python (<code>AppointmentService</code>); мёртвая схема только вводила в заблуждение.

Миграция	Что меняет
0006_users_is_blocked	Добавляет <code>users.is_blocked</code> – блокировка аккаунта (не мягкое удаление: аккаунт и история остаются, но вход запрещён).
0007_users_token_version	Добавляет <code>users.token_version</code> – отзыв ранее выданных JWT (раздел 10).
0008_users_vk_oauth	<code>password_hash</code> становится необязательным, добавляется <code>vk_user_id</code> – вход через VK ID создаёт пользователей без пароля.
0009_sessions	Добавляет таблицу <code>sessions</code> (refresh-токены на отдельное устройство) – <code>logout</code> начинает удалять ровно одну сессию вместо инвалидации всех токенов сразу.
0010_appointment_reminders	Добавляет <code>appointments.reminder_24h_sent_at/reminder_2h_sent_at</code> – напоминания точно на самой записи, а не отдельной таблицей уведомлений (та уже была и её снесли в 0005).
0011_users_email_nullable	<code>users.email</code> становится необязательным – VK ID не всегда отдаёт email при регистрации.
0012_email_verification	Добавляет <code>users.email_verified_at</code> и таблицу <code>email_verification_tokens</code> – подтверждение email тем же паттерном, что и сброс пароля (хеш токена в БД, сам токен – в письме).

Миграция	Что меняет
0013_salons	Первый шаг функционала «сеть точек»: создаёт таблицу <code>salons</code> и переносит уже сохранённые данные <code>site_settings.content</code> в одну строку-заглушку («Салон №1»). Поведение приложения ещё не меняется – таблицу пока никто не читает.
0014_user_role_owner	Единственная строка: добавляет значение <code>'owner'</code> в <code>enum user_role</code> . Намеренно отдельная миграция – PostgreSQL запрещает использовать значение <code>enum</code> , добавленное <code>ALTER TYPE ... ADD VALUE</code> , в той же транзакции, где оно добавлено; следующая миграция уже увидит <code>'owner'</code> закоммитенным.
0015_salon_scoping	Добавляет <code>salon_id</code> пользователям (обязателен только для <code>admin</code>) и записям, повышает всех существующих <code>admin</code> до <code>owner</code> . Обязана деплоиться одновременно с обновлённым <code>get_current_admin/require_role</code> (раздел 10) – иначе только что повышенные владельцы мгновенно теряют доступ к <code>/api/admin/*</code> до раскатки нового кода.

11.3 Двойное бронирование невозможно физически, а не по вежливой просьбе

Раздел 3.3 уже показал, где эта защита стоит в потоке кода. Вот её фактический SQL, добавленный вручную в `0001_initial` (SQLAlchemy не умеет сгенерировать такое ограничение автоматически, поэтому это единственный кусок сырого SQL в первой миграции):

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER TABLE appointments
ADD CONSTRAINT no_double_booking
EXCLUDE USING gist (
    master_id WITH =,
    tstzrange(start_time, end_time, '[]') WITH &&
)
WHERE (status <> 'cancelled')
```

Своими словами: «для одного и того же `master_id` никакие два временных диапазона `[start_time, end_time)` не могут пересекаться – за исключением строк с `status = 'cancelled'`». `EXCLUDE USING gist` – ограничение целостности PostgreSQL уровня таблицы, того же семейства, что и обычный `UNIQUE`, но проверяющее не равенство значений, а пересечение диапазонов (`tstzrange` – диапазон момента времени с часовым поясом). `WHERE (status <> 'cancelled')` – частичное ограничение: отменённая запись не блокирует слот навсегда, к нему снова можно записаться. Это единственное место во всём проекте, где требование «0 инцидентов двойного бронирования» гарантируется *в принципе*, а не только проверкой в коде приложения перед вставкой строки, которая при параллельных запросах подвержена состоянию гонки – сравнение с предварительной проверкой на уровне сервиса (шаг 5 раздела 3.3) и приёмочный тест на настоящем Postgres – раздел 12.

11.4 backend/schema.sql – читаемый снимок, не источник истины

В корне `backend/` рядом с `alembic/` лежит ещё один файл – `schema.sql`, обычный SQL-дамп «как выглядит вся схема целиком». Он сам предупреждает о своей роли в собственном заголовке:

```
-- Сгенерировано по факту применения всех Alembic-миграций
-- (backend/alembic/versions/0001..0007) – это снимок итоговой
↪ схемы,
-- а не замена миграций. Для реального развёртывания используйте
-- `alembic upgrade head`; этот файл – читаемый обзор структуры
↪ БД,
-- пригодный для импорта в dbdiagram.io (Import from SQL →
↪ PostgreSQL).
```

Важная деталь для новичка: комментарий `honestly` говорит «0001..0007» – файл описывает состояние схемы *после седьмой* миграции, а не после пятнадцатой. Он не обновлялся при добавлении `sessions`, подтверждения `email` и всего функционала сети точек. Единственный источник правды о текущей схеме БД – цепочка миграций в `alembic/versions/` (или сама работающая база данных); этот файл пригоден только для быстрого визуального обзора *ранней* версии структуры, не для реальной работы с проектом.

12 Тесты бэкенда

12.1 Два принципиально разных набора тестов

`backend/tests/conftest.py` прямо в докстринге объясняет главное архитектурное решение тестов проекта:

```
"""Юнит-тесты без БД: сервисы получают фейковые репозитории,  
→ схемы и чистые  
функции тестируются напрямую. Интеграционных тестов с PostgreSQL  
→ нет —  
ключевая защита от двойного бронирования на уровне БД (EXCLUDE  
→ USING gist)  
проверяется вручную/на стенде."""
```

Это было верно на момент написания докстринга — позже в проекте появился второй набор, `backend/tests/integration/`, именно для того, что этот комментарий называет «вручную/на стенде» (докстринг с тех пор не обновляли, что само по себе неплохой урок — комментарии в коде могут устаревать, полагаться в первую очередь стоит на то, что реально запускается в CI, раздел 19).

Юнит-тесты (`backend/tests/*.py`, около двадцати файлов) запускаются командой `pytest -q` без каких-либо внешних зависимостей — быстро, без базы данных вообще. Сервис в тесте получает не настоящий `UserRepository/AppointmentRepository`, а простой Python-объект с тем же набором методов, хранящий данные в обычном словаре или списке в памяти — «фейковый репозиторий» (fake, не mock-библиотека). Это возможно именно потому, что сервисы (раздел 7) принимают сессию БД через конструктор, а не создают подключение сами — граница между слоями, которую легко подменить в тесте.

Интеграционные тесты (`backend/tests/integration/*.py`, семь файлов) запускаются отдельной командой на настоящем PostgreSQL:

```
pytest -m integration tests/integration -q
```

и требуют переменной окружения `TEST_DATABASE_URL` — без неё весь пакет автоматически пропускается (`pytest.skip(..., allow_module_level=True)` в его `conftest.py`), так что обычный

pytest -q без настроенного Postgres продолжает работать как раньше. backend/pytest.ini явно исключает эту группу из дефолтного прогона (addopts = -m "not integration") – юнит- и интеграционные тесты никогда не запускаются одной и той же командой случайно.

```
[pytest]
markers =
    integration: requires a real Postgres database (see
        ↪ tests/integration/conftest.py)
addopts = -m "not integration"
```

Эти тесты существуют ровно там, где фейковые репозитории структурно не могут ничего проверить – ограничение EXCLUDE (раздел 11.3), частичные уникальные индексы, реальные join-запросы репозитория при скоупинге по точке сети, и happy path целиком через настоящий HTTP-стек (httpx.AsyncClient поверх реального ASGI-приложения). Пример – test_double_booking.py, прямое продолжение раздела 3.3 и 11.3:

```
def test_exclusion_constraint_rejects_overlapping_appointment(db_
↪ _session, bookable_setup):
    """Доказывает, что пересечение физически невозможно на уровне
    ↪ БД, а не
    только что сервис вежливо отказывает при последовательных
    ↪ запросах."""
    ...
    with pytest.raises(IntegrityError) as exc_info:
        repo.create(client_b.id, AppointmentCreate(...,
            ↪ start_time=overlap_start), ...)
    assert exc_info.value.orig.diag.constraint_name ==
    ↪ "no_double_booking"
```

```
@pytest.mark.anyio
async def
↪ test_concurrent_booking_requests_only_one_succeeds(client,
↪ db_session, bookable_setup):
    """Две ОДНОВРЕМЕННЫЕ HTTP-заявки на один и тот же слот."""
    responses = await asyncio.gather(*(
        client.post("/api/appointments", json=payload,
            ↪ headers={"Authorization": f"Bearer {token}"})
```

```

        for token in tokens
    ))
    statuses = sorted(r.status_code for r in responses)
    assert statuses == [201, 409], f"unexpected statuses:
    ↪ {[r.text for r in responses]}"

```

Второй тест – буквально то же состояние гонки, о котором предупреждает раздел 3.3 (шаг 6), воспроизведённое по-настоящему: `asyncio.gather` отправляет два запроса на бронирование одного и того же слота одновременно и проверяет, что ровно один получает 201, а второй – 409 (и никогда не 500).

12.2 Каталог тестов

Таблица 5 – backend/tests/ – юнит-тесты (без БД)

Файл	Что проверяет
<code>test_auth_unit.py</code>	Хеширование/проверка паролей и RBAC-зависимости (<code>require_role</code>).
<code>test_admin_role_assignment.py</code>	<code>AdminService</code> : сетевая иерархия ролей – назначить <code>admin/owner</code> может только <code>owner</code> .
<code>test_admin_session_revocation.py</code>	События безопасности (смена пароля, блокировка, удаление) отзывают и <code>token_version</code> , и все <code>refresh</code> -сессии разом.
<code>test_admin_master_profile.py</code>	Реактивация профиля мастера вместо создания нового при повторном назначении роли.
<code>test_appointment_create_guard.py</code>	Лимит активных записей клиента и запрет самозаписи мастера к себе.
<code>test_appointment_reschedule.py</code>	<code>AppointmentService.reschedule()</code> .
<code>test_business_hours.py</code>	Часы работы точки как жёсткая граница и для расписаний мастеров, и для самих записей.

Файл	Что проверяет
<code>test_client_ip.py</code>	<code>client_ip()</code> – реальный IP через цепочку прокси Caddy → nginx.
<code>test_content_filter.py</code>	<code>contains_profanity()/contains_link()</code> – в первую очередь ложные срабатывания фильтра.
<code>test_email_verification.py</code>	Методы подтверждения email в AuthService.
<code>test_health.py</code>	GET /health возвращает 503, если БД недоступна – не только «процесс жив».
<code>test_rate_limit.py</code>	Подключение лимита частоты запросов и его обработчик исключения.
<code>test_reminder_service.py</code>	<code>ReminderService.send_due_reminders()</code> .
<code>test_review_schema.py</code>	Цензура/запрет ссылок в отзыве применяются на входе схемы, до создания.
<code>test_salon_scoping.py</code>	<code>resolve_salon_scope</code> – ветвления по роли на фейковых репозиториях.
<code>test_schemas.py</code>	Валидация схем: пароли (лимит bcrypt), расписание, таймзоны, деньги.
<code>test_setup_service.py</code>	Первичная настройка (создание владельца сети + первой точки) на фейковом репозитории.
<code>test_slots.py</code>	Генерация свободных слотов бронирования.
<code>test_slug.py</code>	<code>slugify()/generate_unique_slug()</code> – генерация URL-слага точки сети.
<code>test_state_machine.py</code>	Машина состояний записи (ALLOWED_TRANSITIONS) и права на переходы.
<code>test_uploads.py</code>	<code>save_uploaded_image()</code> – тип/размер файла и что он реально попадает на диск под сгенерированным именем.

Файл	Что проверяет
test_vk_oauth_service.py	VK ID OAuth (PKCE) – без сети и БД, фейковый репозиторий и подмена <code>httpx.post</code> .

Таблица 6 – backend/tests/integration/ – на реальном Postgres

Файл	Что проверяет
test_double_booking.py	Ограничение EXCLUDE USING gist и настоящая гонка двух одновременных запросов (раздел 11.3).
test_soft_delete_constraints.py	Частичные уникальные индексы (WHERE deleted_at IS NULL) реально освобождают email/телефон/vk_user_id при мягком удалении.
test_salon_scoping_flow.py	Скоупинг по точке сети сквозь настоящий HTTP-стек и join-запросы репозитория.
test_salon_slug_uniqueness.py	SalonService.create() разрешает коллизию slug суффиксом через реальные повторные запросы к БД.
test_admin_role_assignment_flow.py	Полный HTTP-флоу: создать пользователя → назначить точку → повысить роль.
test_fresh_install_setup.py	Первичная настройка на пустой БД – регрессия ограничения ck_users_admin_requires_salon.
test_happy_path_e2e.py	Полный сценарий целиком: регистрация → свободный слот → бронирование → подтверждение → завершение – через реальный HTTP-стек и БД.

13 Frontend: точка входа и маршрутизация

13.1 main.js – где всё начинается

`frontend/src/main.js` – первый файл, который выполняется в браузере. Он создаёт приложение Vue, подключает Pinia и роутер, и настраивает две вещи, которых нет в типичном учебном примере Vue-проекта:

```
const app = createApp(App)
app.use(createPinia())
app.use(router)

setUnauthorizedHandler(() => {
  const auth = useAuthStore()
  if (!auth.isLoggedIn) return
  auth.clearSession()
  const current = router.currentRoute.value
  if (current.path !== '/login') {
    router.push({ path: '/login', query: { redirect:
      ↪ current.fullPath } })
  }
})

app.config.errorHandler = (err, _instance, info) => {
  console.error('[vue error]', info, err)
  useToastStore().error('Что-то пошло не так. Обновите
    ↪ страницу.')
}

if (import.meta.env.VITE_SENTRY_DSN) {
  Sentry.init({ app, dsn: import.meta.env.VITE_SENTRY_DSN,
    ↪ tracesSampleRate: 0 })
}

app.mount('#app')
```

`setUnauthorizedHandler(...)` – та самая функция, которую вызовет `api/client.js` (раздел 15), когда сервер окончательно отвергнет токен (после неудачной попытки обновить его через `refresh`). Она объявлена именно здесь, а не внутри самого `client.js`, чтобы избежать циклического импорта: `client.js` → `router` → `stores` → `client.js`.

`app.config.errorHandler` – глобальный перехватчик необработанных ошибок рендера Vue-компонентов («error boundary») – без него исключение в шаблоне молча ломает страницу; здесь оно вместо этого показывает тост и пишет в консоль. `VITE_SENTRY_DSN` не задан по умолчанию (то же самое, что `SENTRY_DSN` на backend, раздел 3.1) – пока переменная пуста, `Sentry.init(...)` не вызывается вовсе, и весь пакет `@sentry/vue` даже не попадает в собранный бандл благодаря tree-shaking.

13.2 App.vue – корневой компонент

`frontend/src/App.vue` – то, что реально монтируется в `#app`. Он тонкий: показывает баннер неподтверждённого email, шапку и подвал сайта (кроме страниц с `meta.hideHeader` – у визарда настройки, кабинета мастера и админки есть свой макет), сам маршрут (`<router-view />`) и контейнер тостов; при монтировании загружает контент сайта и список точек сети – они нужны публичным экранам (футер, секция «Наши салоны», фильтр каталога), поэтому грузятся здесь, а не только внутри админки.

13.3 router/index.js – все маршруты в одном файле

Весь список маршрутов SPA – один файл, `frontend/src/router/index.js`. Каждый компонент страницы подключён через `() => import(...)` – ленивая загрузка: код конкретной страницы попадает в браузер отдельным чанком только когда пользователь на неё переходит, а не сразу при открытии сайта.

```
const routes = [
  { path: '/setup', name: 'setup', component: () =>
    ↪ import('../views/SetupWizard.vue'),
    meta: { hideHeader: true } },
  { path: '/', name: 'home', component: () =>
    ↪ import('../views/HomePage.vue') },
  { path: '/masters/:id', name: 'master-booking', component: ()
    ↪ => import('../views/BookingPage.vue') },
  { path: '/login', ..., meta: { guestOnly: true } },
  { path: '/profile', ..., meta: { requiresAuth: true } },
  {
```

```

    path: '/dashboard',
    component: () => import('../views/MasterDashboard.vue'),
    meta: { requiresAuth: true, roles: ['master'], hideHeader:
      ↪ true },
    children: [ /* dashboard-appointments, dashboard-schedule,
      ↪ dashboard-reviews */ ],
  },
  {
    path: '/admin',
    component: () => import('../views/AdminPanel.vue'),
    meta: { requiresAuth: true, roles: ['admin', 'owner'],
      ↪ hideHeader: true },
    children: [ /* admin-stats, admin-users, admin-services,
      ↪ admin-masters, ... */ ],
  },
  { path: '/*:pathMatch(.*)*', name: 'not-found', component: () =>
    ↪ import('../views/NotFoundPage.vue') },
]

```

Обратите внимание на маршрут `/admin/salons` внутри детей `/admin` – у него собственный `meta.roles: ['owner']`, переопределяющий более широкий `['admin', 'owner']` родителя: точками сети может управлять только владелец сети, хотя саму админку видит и обычный администратор точки.

Единственный глобальный хук `router.beforeEach` – это то место, где решается, пускать ли вообще на запрошенную страницу:

```

router.beforeEach(async (to) => {
  const setup = useSetupStore()
  if (!setup.checked) await setup.checkStatus()
  if (!setup.completed) {
    return to.name === 'setup' ? true : { name: 'setup' }
  }
  if (setup.completed && to.name === 'setup') return { name:
    ↪ 'home' }

  const auth = useAuthStore()
  if (!auth.ready) await auth.fetchMe()

  if (to.meta.requiresAuth && !auth.isLoggedIn) {
    return { name: 'login', query: { redirect: to.fullPath } }
  }

```

```

    }
    if (to.meta.roles && !to.meta.roles.includes(auth.user?.role))
      ↪ {
        return { name: 'home' }
      }
    if (to.meta.guestOnly && auth.isLoggedIn) {
      return (typeof to.query.redirect === 'string' &&
        ↪ to.query.redirect) || { name: 'home' }
    }
    return true
  })

```

Порядок проверок здесь принципиален. Первым делом хук спрашивает `useSetupStore()`, завершена ли первичная настройка системы (раздел 8, GET `/api/setup/status`) – если нет, *любой* маршрут, кроме самого `/setup`, принудительно перенаправляется на визард: обойти мастер первого запуска, зная прямой URL другой страницы, невозможно. Дальше – обычная связка «дождаться, кто я» (`auth.fetchMe()`, раздел 14) и «имею ли право сюда попасть» по `meta.requiresAuth/meta.roles`. Раз запрошенный так и не выполненный `auth.fetchMe()` значит, что проверка `await` гарантированно дожидается ответа сервера прежде, чем решение о доступе будет принято – иначе на быстром интернете защищённая страница на мгновение мигнула бы для неавторизованного пользователя.

14 Frontend: управление состоянием (Pinia stores)

Все хранилища в `frontend/src/stores/` написаны в *setup-стиле* Pinia – то есть выглядят как обычная Vue Composition API функция (`ref`, `computed`), а не как объект с полями `state/getters/actions` (более старый, Options-стиль Pinia). Vuex в проекте не используется вовсе.

14.1 `stores/auth.js` – кто сейчас залогинен

Самое часто используемое хранилище проекта. Его центральная идея – намеренное отсутствие токена внутри:

```
export const useAuthStore = defineStore('auth', () => {
  const user = ref(null)
  const ready = ref(false)

  const isLoggedIn = computed(() => !!user.value)
  const isOwner = computed(() => user.value?.role === 'owner')
  const isAdmin = computed(() => user.value?.role === 'admin' ||
    → isOwner.value)

  async function fetchMe() {
    try {
      const res = await authApi.me()
      user.value = res.data
    } catch (err) {
      if (err.response?.status === 401) clearSession()
    } finally {
      ready.value = true
    }
  }

  function clearSession() {
    user.value = null
  }

  return { user, ready, isLoggedIn, isClient, isMaster, isAdmin,
    → isOwner, salon,
      register, login, fetchMe, updateMe, logout,
    → clearSession }
}
```

})

Токен живёт только в httpOnly-cookie (раздел 10) – JavaScript физически не может его прочитать, поэтому хранилище вообще не пытается. Сессия на фронтенде определяется исключительно тем, есть ли объект `user` или он `null`; чтобы узнать, залогинен ли вообще посетитель при первой загрузке страницы, единственный способ – спросить сервер (GET `/api/auth/me`) – отсюда `fetchMe()`, которую ждёт `router.beforeEach` перед первой навигацией. Обратите внимание, что `isAdmin` явно включает `isOwner.value` – то же правило «owner $\supset$ admin» из раздела 1 и 10, продублированное на фронтенде для условий в шаблонах.

14.2 Остальные хранилища

frontend/src/stores/toast.js Всплывающие уведомления – единая точка показа сообщений об успехе, ошибке или информации в любом месте приложения.

```
toasts                # ref, массив активных тостов
push(message, variant = 'info', duration = 4000)  # добавляет
  ↳ тост, планирует автоскрытие
dismiss(id)
success(message)      # push(..., 'success')
error(message)        # push(..., 'error')
info(message)
```

frontend/src/stores/setup.js Статус визарда первого запуска – определяет, нужно ли показать пользователю страницу `/setup` вместо обычных страниц сайта (раздел 13.3).

```
completed            # ref, по умолчанию true
requiresToken        # ref
checked              # ref
checkStatus()        # запрашивает setupApi.status(); при сетевой
  ↳ ошибке считает настройку завершённой
markCompleted()      # принудительно помечает настройку завершённой
  ↳ после успешного /setup
```

Особенности: `completed` по умолчанию `true` – намеренное «fail-open» поведение: сетевая ошибка при проверке статуса не должна блокировать доступ

к уже настроенному сайту.

frontend/src/stores/salon.js Список точек сети (салонов). Грузится один раз при старте приложения (в `App.vue`) и используется переключателем точки в админке, футером сайта и секцией «Наши салоны» на главной.

```
salons          # ref, полный список точек
viewingSalonId  # ref, id точки, которую сейчас просматривает
  ↳ owner; null = «вся сеть»
activeSalons    # computed, только активные (открытые) точки
byId           # computed, словарь точек по id
viewingSalon    # computed
load(force = false)
setViewing(id)
refresh()       # load(true)
```

Особенности: GET `/api/salons` отдаёт разный список в зависимости от роли (закрытые точки видят только `admin/owner`), поэтому `AdminPanel.vue` перезапрашивает список с `force=true` при входе в панель, а публичные экраны читают `activeSalons` и остаются корректными даже с расширенным списком в сторе.

frontend/src/stores/masterProfile.js Кэширует профиль мастера текущего авторизованного пользователя. Нужен во всех вкладках личного кабинета мастера (записи, расписание, отзывы), чтобы каждая под-страница не запрашивала профиль заново.

```
profile         # ref
loaded          # ref
load(force = false) # через mastersApi.getMe(); повторный вызов
  ↳ без force пропускается
```

frontend/src/stores/siteContent.js Редактируемый через CMS (`/admin/settings`) контент сайта – тексты шапки, hero-блока, секций услуг/мастеров/СТА, футера, SEO-метаданные и тема оформления. Значения по умолчанию дублируют backend-дефолты, поэтому публичные страницы выглядят правильно ещё до первого ответа сервера.

```
content          # ref
load(force = false) # через settingsApi.get(), затем
↳ applyTheme/applyFont/applySeo
set(data)         # заменяет контент целиком (после сохранения в
↳ редакторе)
```

Особенности: `applyTheme/applyFont/applySeo` (раздел 18) вызываются уже в момент создания стора (на дефолтных значениях) и при каждой загрузке/замене – тема и SEO-теги применяются к живому DOM, а не только на экране редактирования.

15 Frontend: слой API (api/)

15.1 api/client.js – один axios на весь фронтенд

Все остальные файлы api/ используют один и тот же настроенный экземпляр axios из frontend/src/api/client.js:

```
const client = axios.create({
  baseURL: '/api',
  headers: { 'Content-Type': 'application/json' },
  withCredentials: true,
})

let refreshPromise = null

function refreshAccessToken() {
  if (!refreshPromise) {
    refreshPromise = axios.post('/api/auth/refresh', null, {
      ↪ withCredentials: true })
      .finally(() => { refreshPromise = null })
  }
  return refreshPromise
}

client.interceptors.response.use(
  (response) => response,
  async (error) => {
    const { config, response } = error
    const isNoRetryEndpoint = NO_RETRY_PATHS.some((path) =>
      ↪ config?.url?.startsWith(path))

    if (response?.status === 401 && config &&
      ↪ !config._retriedAfterRefresh && !isNoRetryEndpoint) {
      config._retriedAfterRefresh = true
      try {
        await refreshAccessToken()
        return client(config)
      } catch { /* refresh тоже не удался – падаем ниже на
        ↪ onUnauthorized */ }
    }
    if (error.response?.status === 401 && onUnauthorized)
      ↪ onUnauthorized()
```

```

    return Promise.reject(error)
  }
)

```

`withCredentials: true` – обязательное условие, чтобы браузер вообще прикладывал `httpOnly-cookie` к запросу. Интересная часть – `refreshPromise`: если сразу несколько запросов на странице получают 401 одновременно (например, несколько виджетов грузят данные параллельно), они не отправят несколько параллельных `POST /api/auth/refresh` – второй и последующие вызовы `refreshAccessToken()` переиспользуют уже начатый общий `promise` вместо создания нового. Это называется *single-flight* и напрямую связано с ротацией `refresh`-токена на backend (раздел 10): если бы два запроса на обновление ушли параллельно по-настоящему, второй получил бы 401 на уже использованный (ротированный первым) `refresh`-токен. `config._retriedAfterRefresh` гарантирует, что повторная попытка происходит не более одного раза на исходный запрос – без этого флага бесконечно повторяющийся 401 после неудачного `refresh` зациклил бы запросы.

15.2 Остальные файлы `api/`

Все перечисленные ниже файлы устроены одинаково – тонкая функция на каждый эндпоинт backend, без собственной логики внутри (сравните с таблицами роутов в разделе 8 – один файл `api/x.js` почти всегда соответствует одному файлу `routes/x.py`).

`frontend/src/api/index.js` Единая точка входа (`barrel`-файл) для всех `api`-модулей и общего HTTP-клиента – остальной код импортирует конкретные `*Api` объекты из `../api`, а не обращается к отдельным файлам напрямую.

`frontend/src/api/auth.js`

```

register(data) -> POST /auth/register
login(data) -> POST /auth/login
logout() -> POST /auth/logout

```

```
me() -> GET /auth/me
updateMe(data) -> PATCH /auth/me
requestPasswordReset(email) -> POST /auth/password-reset/request
confirmPasswordReset(token, newPassword) -> POST
  ↳ /auth/password-reset/confirm
confirmEmailVerification(token) -> POST
  ↳ /auth/email-verification/confirm
resendEmailVerification() -> POST /auth/email-verification/resend
vkEnabled() -> GET /auth/vk/enabled
```

frontend/src/api/appointments.js

```
create(data) -> POST /appointments
getById(id) -> GET /appointments/{id}
cancel(id) -> POST /appointments/{id}/cancel
updateStatus(id, statusValue) -> PATCH /appointments/{id}/status
reschedule(id, startTime) -> PATCH /appointments/{id}/reschedule
listMy(params) -> GET /appointments/my
listForMaster(masterId, params) -> GET
  ↳ /appointments/master/{masterId}
listAll(params) -> GET /appointments
```

frontend/src/api/masters.js

```
list(params) -> GET /masters
getMe() -> GET /masters/me
getById(id) -> GET /masters/{id}
update(id, data) -> PATCH /masters/{id}
getServices(id) -> GET /masters/{id}/services
addService(id, serviceId, priceOverride) -> POST
  ↳ /masters/{id}/services
removeService(id, serviceId) -> DELETE
  ↳ /masters/{id}/services/{serviceId}
getSchedule(id) -> GET /masters/{id}/schedule
setSchedule(id, data) -> POST /masters/{id}/schedule
updateSchedule(id, dayOfWeek, data) -> PATCH
  ↳ /masters/{id}/schedule/{dayOfWeek}
getSlots(id, serviceId, targetDate, excludeAppointmentId) -> GET
  ↳ /masters/{id}/slots
```

Особенности: `getSlots` принимает необязательный `excludeAppointmentId` – нужен при переносе существующей записи, чтобы её же текущий слот не считался занятым при расчёте доступности.

frontend/src/api/salons.js

```
list(params) -> GET /salons
getById(id) -> GET /salons/{id}
create(data) -> POST /salons
update(id, data) -> PATCH /salons/{id}
```

frontend/src/api/services.js

```
list(params) -> GET /services
getById(id) -> GET /services/{id}
create(data) -> POST /services
update(id, data) -> PATCH /services/{id}
```

frontend/src/api/reviews.js

```
create(data) -> POST /reviews
listForMaster(masterId, params) -> GET /reviews/master/{masterId}
listAll(params) -> GET /reviews
moderate(id, isPublished) -> PATCH /reviews/{id}/publish
remove(id) -> DELETE /reviews/{id}
```

frontend/src/api/admin.js

```
getStats(params) -> GET /admin/stats
listUsers(params) -> GET /admin/users
createUser(data) -> POST /admin/users
updateUser(userId, data) -> PATCH /admin/users/{userId}
changeUserRole(userId, role) -> PATCH /admin/users/{userId}/role
assignUserSalon(userId, salonId) -> PATCH
  ↳ /admin/users/{userId}/salon
setUserBlocked(userId, isBlocked) -> PATCH
  ↳ /admin/users/{userId}/block
createMasterProfile(userId, salonId) -> POST
  ↳ /admin/users/{userId}/master
deleteUser(userId) -> DELETE /admin/users/{userId}
deleteService(serviceId) -> DELETE /admin/services/{serviceId}
updateMasterPhoto(masterId, photoUrl) -> PATCH
  ↳ /admin/masters/{masterId}/photo
uploadImage(file) -> POST /admin/uploads/image # тело – FormData
getReport(params) -> GET /admin/reports
exportReport(params) -> GET /admin/reports/export #
  ↳ responseType: 'blob'
```

Особенности: assignUserSalon обязана вызываться раньше changeUserRole(userId, 'admin') – ограничение БД ck_users_admin_requires_salon требует, чтобы точка уже была назначена в момент повышения роли до администратора (раздел 10).

frontend/src/api/settings.js

```
get() -> GET /settings  
update(data) -> PATCH /settings
```

frontend/src/api/setup.js

```
status() -> GET /setup/status  
complete(data) -> POST /setup
```

16 Frontend: компоненты

16.1 components/ui/ – переиспользуемые «кирпичики»

Самый мелкий уровень интерфейса – кнопка, поле ввода, карточка. Ни один файл здесь не знает о бизнес-логике приложения, только о том, как выглядеть и что эмитить через `v-model`. Разберём один целиком, чтобы дальше читать остальные по аналогии.

`frontend/src/components/ui/BaseButton.vue` – самый простой и самый часто используемый компонент проекта:

```
<template>
  <button :type="type" :disabled="disabled || loading"
    class="... font-mono uppercase tracking-wider ..."
    :class="[variants[variant] ?? variants.primary, sizes[size]
      ↪ ?? sizes.md]">
    <svg v-if="loading" class="h-4 w-4 animate-spin"
      ↪ ...>...</svg>
    <slot />
  </button>
</template>

<script setup>
defineProps({
  variant: { type: String, default: 'primary' },
  size: { type: String, default: 'md' },
  loading: Boolean,
  disabled: Boolean,
  type: { type: String, default: 'button' },
})
const variants = { primary: '...', accent: '...', ghost: '...',
  ↪ danger: '...' }
const sizes = { sm: '...', md: '...', lg: '...' }
</script>
```

Три вещи, которые повторяются в десятках других компонентов проекта: `<slot />` – содержимое кнопки передаёт вызывающий код (`<BaseButton>Записаться</BaseButton>`), сам компонент не решает, какой текст показать; проп `loading` сам подставляет крутящийся спиннер и блокирует повторный клик – вызывающему коду достаточно

привязать один `ref` к состоянию запроса; словари `variants/sizes` – готовый набор вариантов оформления вместо произвольных классов на каждом месте использования.

Чуть сложнее – `frontend/src/components/ui/ImageUpload`. `.vue`: показывает превью картинки и сам делает запрос загрузки на сервер при выборе файла:

```
async function onChange(event) {
  const file = event.target.files?.[0]
  event.target.value = ''
  if (!file) return
  uploading.value = true
  try {
    const { data } = await adminApi.uploadImage(file)
    emit('update:modelValue', data.url)
  } catch (err) {
    toast.error(extractErrorMessage(err, 'Не удалось загрузить
    ↳ изображение'))
  } finally {
    uploading.value = false
  }
}
```

Компонент реализует `v-model` через связку пропа `modelValue` и события `update:modelValue` – стандартный для Vue приём, встречающийся почти во всех `ui/-`компонентах ниже: родитель просто пишет `<ImageUpload v-model="master.photo_url" />` и не заботится, как именно внутри происходит загрузка на `POST /api/admin/uploads/image` (раздел 15) и получение готового URL обратно.

Таблица 7 – остальные 18 файлов `components/ui/`, все в том же духе: маленький, самодостаточный, переиспользуемый в нескольких местах.

Таблица 7 – `components/ui/` – остальные компоненты

Файл	Назначение
<code>BaseCard.vue</code>	Базовая карточка-контейнер (рамка + паддинг) – универсальный «холст» для контента по всей админке и кабинету мастера.

Файл	Назначение
BaseCheckbox.vue	Стилизованный чекбокс с <code>v-model</code> – настоящий <code><input type="checkbox"></code> остаётся в разметке (скрыт визуально), видимая галочка – отдельная иконка поверх него, чтобы сохранить нативные фокус/доступность.
BaseInput.vue	Базовое поле формы (<code>input</code> или <code>textarea</code>) с лейблом, подсказкой и текстом ошибки – основной элемент ввода почти во всех формах приложения.
BaseSelect.vue	Базовый выпадающий список с лейблом и собственной нарисованной стрелкой вместо нативной.
BaseDateInput.vue	Кастомный календарь выбора одной даты – замена нативного <code><input type="date"></code> там, где нужен гарантированно русскоязычный интерфейс (нативный виджет следует локали браузера/ОС, может показать формат <code>mm/dd/yyyy</code> даже при <code>lang="ru"</code> на странице).
BaseTimeInput.vue	Выбор времени двумя выпадающими списками (часы/минуты) – замена нативного <code><input type="time"></code> , который может показывать 12-часовой формат АМ/ПМ в зависимости от локали ОС.
ConfirmDialog.vue	Универсальное модальное окно подтверждения опасного действия (например удаления) – используется по всей админке в списках.

Файл	Назначение
<code>EditableText.vue</code>	Текст, который в обычном режиме – статичный тег, а в режиме редактирования сайта превращается в поле, редактируемое прямо на странице – основной строительный блок инлайн-редактирования контента главной страницы владельцем.
<code>EditableLink.vue</code>	Ссылка-обёртка, которая в режиме редактирования контента превращается из кликабельной ссылки в обычный <code></code> – чтобы клики по СТА-кнопкам на главной не уводили редактора с текущей страницы.
<code>EmptyState.vue</code>	Заглушка «здесь пока ничего нет» – вместо списков/таблиц, когда данных нет.
<code>HeroPhotoUpload.vue</code>	Оверлей с кнопками «Выбрать»/«Убрать» поверх фото в hero-блоке настроек сайта – та же логика загрузки, что и у <code>ImageUpload.vue</code> выше.
<code>Pagination.vue</code>	Постраничная навигация «Назад/Далее» со счётчиком «Страница X из Y» – под списками/таблицами админки; полностью скрывает себя, если страница всего одна.
<code>Skeleton.vue</code>	Прямоугольник-заглушка с пульсирующей анимацией – плейсхолдер на время загрузки контента.
<code>StarRatingInput.vue</code>	Выбор оценки от 1 до 5 звёзд кликом – в форме отзыва клиента; размечен как <code>role="radiogroup"</code> для доступности.
<code>StatusPill.vue</code>	Цветной бейдж статуса записи (ожидает / подтверждена / завершена / отменена) – везде, где показывается <code>appointment.status</code> .

Файл	Назначение
StepTitle.vue	Заголовок пронумерованного шага в пошаговой форме бронирования (views/BookingPage.vue, см. раздел 17).
ToastContainer.vue	Контейнер всплывающих уведомлений в правом нижнем углу экрана – подключён один раз в App.vue (раздел 13.2), читает состояние из useToastStore (раздел 14).
VkLoginButton.vue	Кнопка «Войти через VK» на страницах входа и регистрации. Клик – полная навигация браузера (window.location.href = . . .), а не axios-запрос: серверу нужно выставить httpOnly-cookie и сделать редирект на id.vk.com (раздел 10), что может сделать только сам браузер.

16.2 components/ – составные компоненты

Крупнее, чем ui/ – собирают несколько «кирпичиков» вместе под конкретную задачу интерфейса и обычно завязаны на конкретные хранилища/API.

Таблица 8 – components/ – составные компоненты

Файл	Назначение
AppHeader.vue	Закреплённая сверху шапка сайта – логотип, навигация (пункты по роли пользователя) и блок авторизации. Тот же компонент используется и на публичных страницах, и в редакторе главной (раздел 18).
AppFooter.vue	Подвал сайта – бренд, редактируемый tagline, список открытых точек сети, соцссылки, copyright и ссылка на страницу политики обработки персональных данных.

Файл	Назначение
DashboardLayout.vue	Общий каркас для авторизованных разделов (кабинет мастера и админка) – боковое меню, мобильное меню, верхняя панель с заголовком, переключателем точки сети (только для владельца) и слот для контента конкретной страницы.
SidebarLink.vue	Один пункт меню боковой панели DashboardLayout.vue – иконка, подпись, ссылка с подсветкой активного пункта.
SalonSwitcher.vue	Переключатель точки сети в шапке DashboardLayout.vue. Виден только владельцу сети (выпадающий список всех точек); администратор одной точки вместо селекта видит статичную подпись – бэкенд (resolve_salon_scope, раздел 10) всё равно принудительно сузит область видимости, поэтому рабочий выбор для него был бы обманом интерфейса.
EmailVerificationBanner.vue	Узкая жёлтая полоса-напоминание о неподтверждённом email с кнопкой повторной отправки письма – подключена один раз в App.vue.
HomePreview.vue	Вся содержательная часть главной страницы – hero-блок (в 3 вариантах вёрстки), бегущая строка услуг, преимущества, услуги и цены, витрина мастеров, точки сети, финальный СТА. Один и тот же компонент рендерит и публичную главную страницу, и её живой предпросмотр в редакторе контента (раздел 18).

Файл	Назначение
<code>Hero3DRoom.vue</code>	Интерактивная 3D-модель интерьера барбершопа для hero-блока – один из вариантов медиа. Модель (около 13 МБ <code>.glb</code> из <code>3d_models/</code>) грузится лениво через <code>IntersectionObserver</code> и плавно поворачивается вслед за курсором мыши.
<code>SalonsSection.vue</code>	Секция «Наши салоны» на главной странице – карточки открытых точек сети с фото, адресом и часами работы.
<code>MasterCard.vue</code>	Карточка мастера с фото, специализацией, именем и ссылкой на страницу записи к нему – в каталоге мастеров и на главной.
<code>AppointmentCard.vue</code>	Карточка одной записи – статус-бейдж, подписи, дата/время, итоговая цена; кнопки действий передаются вызывающей страницей через слот. Используется и в личном кабинете клиента, и в кабинете мастера.
<code>RescheduleModal.vue</code>	Модальное окно переноса записи клиента на другую дату/время – выбор даты с подгрузкой свободных слотов мастера под неё.
<code>KpiCard.vue</code>	Небольшая карточка-метрика (иконка + подпись + значение) для страниц статистики и отчётов админки.
<code>RegistrationsChart.vue</code>	Линейный график регистраций новых пользователей по дням на странице статистики – нарисован вручную на SVG, без сторонней библиотеки графиков (данные продублированы скрытой таблицей для скринридеров).

16.3 composables/ – переиспользуемая логика

Composable – функция, использующая реактивность Vue (`ref`, `watch`), но не привязанная к конкретному компоненту – в отличие от Pinia-хранилищ (раздел 14), не хранит общее состояние приложения, просто переиспользует небольшой кусок логики между компонентами.

frontend/src/composables/useFormErrors.js Хранение и управление ошибками полей формы – реактивный объект `errors` и методы для его изменения; используется в паре с `BaseInput/BaseSelect` (их проп `error`) почти во всех формах приложения.

```
useFormErrors() -> { errors, setError, clearError, clearAll,  
  ↪ setFromResponse }  
setFromResponse(err) # разбирает detail[] из 422-ответа  
  ↪ FastAPI, раскладывает по полям
```

Особенности: `setFromResponse` не занимается переводом сообщений сама – делегирует его функции `friendlyValidationError` из `utils/errors.js` и только раскладывает результат по ключам формы.

frontend/src/composables/useDebounceWatch.js Обёртка над `Vue watch()`, откладывая вызов `callback` до тех пор, пока отслеживаемое значение не перестанет меняться в течение заданного времени (`debounce`). Используется там, где не нужно дёргать API на каждое нажатие клавиши (поиск, фильтры в списках админки).

```
useDebounceWatch(source, callback, delay = 350)
```

frontend/src/utils/errors.js Перевод технических ошибок валидации FastAPI/Pydantic (422) в понятные пользователю сообщения на русском – общая точка входа как для текста под конкретным полем формы, так и для текста в тосте об ошибке.

```
friendlyValidationError(item) -> { field, message }  
extractErrorMessage(err, fallback = 'Что-то пошло не так.  
  ↪ Попробуйте ещё раз.') -> String
```

Особенности: для собственных серверных валидаторов (`raise ValueError( ... )`), разделы 5, 9) сообщение берётся как есть, только обрезается автопрефикс Pydantic `"Value error, "` – считается, что оно уже написано понятным русским текстом.

frontend/src/utils/appointmentStatus.js Константы, описывающие жизненный цикл статуса записи на фронтенде – какие переходы допустимы и как подписаны кнопки действий.

```
ALLOWED_TRANSITIONS      # { pending: [...], confirmed: [...],  
    ↪ done: [], cancelled: [] }  
STATUS_ACTION_LABELS     # { confirmed, done, cancelled } ->  
    ↪ текст кнопки  
RESCHEDULABLE_STATUSES   # ['pending', 'confirmed']
```

Особенности: это зеркало серверной машины состояний (`ALLOWED_TRANSITIONS` в `appointment_service.py`, раздел 3.3 и 7) только для отображения нужных кнопок в интерфейсе – окончательную проверку допустимости перехода всегда делает сервер, а не этот файл.

17 Frontend: страницы (views/)

17.1 BookingPage.vue – пошаговое бронирование

Самая важная страница с точки зрения бизнеса – четыре шага (услуга, дата, время, подтверждение), реализующие ровно тот HTTP-запрос, который уже разобран в разделе 3.3.

```
async function selectDate(iso) {
  selectedDate.value = iso
  slotsLoading.value = true
  try {
    const { data } = await mastersApi.getSlots(route.params.id,
      ↪ selectedService.value.service.id, iso)
    slots.value = data.slots
  } finally {
    slotsLoading.value = false
  }
}

async function book() {
  if (!auth.user?.email && !emailInput.value) {
    toast.error('Укажите email, чтобы оформить запись')
    return
  }
  bookingLoading.value = true
  try {
    if (!auth.user?.email) await auth.updateMe({ email:
      ↪ emailInput.value })
    await appointmentsApi.create({
      master_id: route.params.id,
      service_id: selectedService.value.service.id,
      start_time: selectedSlot.value.start_time,
    })
    booked.value = true
  } finally {
    bookingLoading.value = false
  }
}
```

Проверка `!auth.user?.email` перед отправкой – то же самое правило, что и на backend (раздел 3.3, шаг 1): клиент, зарегистрированный через VK

без email, обязан указать его перед первой записью, и страница сама предлагает поле ввода прямо на шаге подтверждения, а не откатывает пользователя на страницу профиля. Список доступных дат (`availableDates`) строится на 14 дней вперёд по календарю UTC (`d.setUTCDate, d.getUTCDate()`), а не по локальному календарю браузера – согласуется с тем, что расписание точки хранится как «настенные часы» в UTC (раздел 3.3, `_validate_within_schedule`):

```
function formatTime(iso) {
  return new Date(iso).toLocaleTimeString('ru', { hour:
    → '2-digit', minute: '2-digit', timeZone: 'UTC' })
}
```

Тот же приём `timeZone: 'UTC'` встретится ещё в нескольких местах (`AppointmentCard.vue`, `RegistrationsChart.vue`) – запомнив его здесь один раз, дальше он не будет удивлять.

17.2 Остальные страницы

Таблица 9 – `views/` – страницы верхнего уровня (по одной на маршрут)

Файл	Назначение и доступ
<code>HomePage.vue</code>	Публичная главная – тонкая обёртка вокруг <code>HomePreview.vue</code> (раздел 16) с контентом из <code>useSiteContentStore</code> . Доступна всем.
<code>MastersPage.vue</code>	Публичный каталог мастеров: поиск по специализации, фильтры по услуге и точке, сортировка, пагинация. Фильтр по точке двусторонне синхронизирован с <code>query</code> -параметром URL через <code>router.replace</code> (не <code>push</code> , чтобы не засорять историю переходов). Доступна всем.

Файл	Назначение и доступ
LoginPage.vue	Вход по email/паролю с клиентской валидацией, плюс вход через VK ID. Отдельно обрабатывает <code>route.query.error</code> – код ошибки VK-редиректа приходит query-параметром, а не HTTP-статусом, потому что пользователь к этому моменту уже находится в браузере посреди редиректа. Доступна неавторизованным (<code>guestOnly</code>).
RegisterPage.vue	Регистрация нового клиента – имя, фамилия, email, телефон, пароль с подтверждением, обязательное согласие на обработку персональных данных; плюс регистрация/вход через VK ID. Доступна неавторизованным.
PasswordResetPage.vue	Двухрежимная страница: без токена в URL – форма запроса ссылки на email, с токеном – форма ввода нового пароля. Сообщение после запроса ссылки одинаковое независимо от того, зарегистрирован ли email – защита от перебора пользователей (раздел 10).
VerifyEmailPage.vue	Подтверждение email по ссылке из письма – трёхсостоятельный экран (загрузка/успех/ошибка). Доступна и авторизованным, и нет – ссылка может быть открыта в любой сессии браузера.
ProfilePage.vue	Личный кабинет клиента – просмотр/редактирование своих данных, список своих записей с фильтром по статусу, отмена записи, отзыв на завершённую услугу. Раздел записей скрыт для роли <code>master</code> . Доступна авторизованным.

Файл	Назначение и доступ
<code>PrivacyPolicyPage.vue</code>	Статическая страница политики обработки персональных данных (152-ФЗ, раздел 2) – название бренда и адрес подставляются из контента сайта. Раздел реквизитов оператора – намеренная плашка-заглушка с явным предупреждением, что документ не имеет юридической силы до заполнения реальными данными.
<code>NotFoundPage.vue</code>	Страница-заглушка 404 – сообщение об ошибке и кнопка перехода в каталог мастеров.
<code>SetupWizard.vue</code>	Четырёхшаговый визард первого запуска – владелец сети, первая точка, базовые настройки бренда, подтверждение (раздел 8, <code>POST /api/setup</code>). Показывается вместо обычных страниц, пока сайт не настроен (раздел 13.3).
<code>AdminPanel.vue</code>	Каркас админ-панели (на <code>DashboardLayout.vue</code>) – боковое меню (статистика, пользователи, услуги, мастера, точки сети, отзывы, отчёты, настройки), <code>router-view</code> для вложенных страниц ниже. Пункты «Точки сети» и «Настройки» видны только владельцу. Доступна <code>admin/owner</code> .
<code>MasterDashboard.vue</code>	Каркас кабинета мастера – боковое меню (записи, расписание, отзывы), <code>router-view</code> ; ждёт загрузки профиля мастера прежде, чем показать вложенные страницы. Доступна <code>master</code> .

Таблица 10 – views/admin/ – вложенные страницы админ-панели

Файл	Назначение
AdminStats.vue	Главный экран (дашборд) – KPI-плитки (пользователи, клиенты, мастера, услуги, записи, выручка за месяц) и график регистраций за 30 дней.
AdminUsers.vue	Список всех пользователей – поиск, фильтр по роли и точке, создание, смена роли, назначение точки, блокировка, удаление. Повышение до admin – двухшаговая операция (сначала назначить точку, потом сменить роль) из-за ограничения БД ck_users_admin_requires_salon.
AdminServices.vue	Каталог услуг сети – поиск, фильтр по активности; полный CRUD только для владельца (общий актив сети), администратор точки видит список только для чтения.
AdminMasters.vue	Список мастеров текущей/выбранной точки – раскрываемые блоки редактирования привязанных услуг, расписания по дням недели и фото.
AdminSalons.vue	Список точек сети – создание, редактирование, открытие/закрытие. Доступна только владельцу. У точек намеренно нет операции удаления (мастера/записи ссылаются на неё с onDelete=RESTRICT) – только «закрытие».
AdminReviews.vue	Все отзывы сети с фильтрами по статусу публикации и минимальному рейтингу, модерация (публикация/скрытие).

Файл	Назначение
<code>AdminReports.vue</code>	Финансовые и операционные отчёты – KPI, линейный график выручки и столбчатая диаграмма по услугам (ручной SVG, без библиотеки), таблица по мастерам, экспорт в Excel (<code>adminApi.exportReport</code> , раздел 7 – <code>ReportService.export_excel</code>).
<code>AdminSettingsLive.vue</code>	Визуальный live-редактор контента и оформления сайта – редактирование прямо поверх предпросмотра (<code>HomePreview.vue</code> , раздел 16), выбор темы/шрифта/SEO-полей. Изменения применяются к живому DOM сразу, ещё до сохранения, и откатываются при уходе со страницы без сохранения. Только владелец (см. раздел 18).

Таблица 11 – `views/dashboard/` – вложенные страницы кабинета мастера

Файл	Назначение
<code>DashboardAppointments.vue</code>	Список записей клиентов к этому мастеру – фильтр по статусу и датам, смена статуса, перенос записи (кнопки – из общих констант <code>utils/appointmentStatus.js</code> , раздел 16).
<code>DashboardSchedule.vue</code>	Редактирование собственного расписания по дням недели – рабочий день/выходной, время начала и окончания; каждый день сохраняется отдельным независимым запросом.
<code>DashboardReviews.vue</code>	Отзывы клиентов о мастере – только просмотр, без модерации.

18 Frontend: тема оформления

Один и тот же собранный фронтенд обслуживает разные точки сети с разным оформлением без пересборки – этого удаётся достичь тем, что цвета бренда объявлены не фиксированными значениями Tailwind, а CSS custom properties, переключаемыми в рантайме.

`frontend/src/theme/presets.js` задаёт три готовых цветовых пресета и функцию применения:

```
export const THEME_TOKENS = [
  { key: 'brand_900', label: 'Основной тёмный (кнопки, текст)' },
  { key: 'accent_400', label: 'Акцент' },
  ...
]

export const THEME_PRESETS = {
  default: { label: 'Чёрный и бежевый', colors: { brand_900:
    ↪ '#111111', ... } },
  coffee: { label: 'Кофе и карамель', colors: { brand_900:
    ↪ '#2B1B12', ... } },
  slate: { label: 'Графит и бирюза', colors: { brand_900:
    ↪ '#1E2530', ... } },
}

export function applyTheme(colors) {
  const root = document.documentElement
  for (const { key } of THEME_TOKENS) {
    if (!colors[key]) continue
    root.style.setProperty(`--color-${key.replace(/_/g, '-')}` ,
    ↪ hexToRgbTriplet(colors[key]))
  }
}
```

`applyTheme` не подставляет HEX-строку напрямую, а переводит её в триплет чисел "R G B" (`hexToRgbTriplet`) – это устройство Tailwind-переменных: определив CSS-переменную как `--color-brand-900: 17 17 17`, сами утилитные классы Tailwind (`bg-brand-900`, `text-brand-900` и т.д.) подключают её через `rgb(var(--color-brand-900) /<alpha-value>)` – то есть поддерживают ещё и прозрачность (`bg-brand-900/60`), что было бы недоступно, храни функция в пе-

ременной готовую CSS-строку цвета. `matchPreset(colors)` делает обратное – определяет, совпадает ли текущий набор цветов с одним из трёх пресетов, или это уже кастомизированная вручную тема (тогда переключатель в `AdminSettingsLive.vue`, раздел 17, показывает «Свой цвет», а не один из трёх пресетов).

frontend/src/theme/fonts.js Набор из пяти готовых пар шрифтов (заголовочный + моноширинный), настраиваемых в «Админка → Настройки → Тема», плюс функция применения выбранной пары к странице.

```
FONT_PRESETS # ключи golos/archivo/manrope/grotesk/inter, у
↪ каждого { label, display, mono, href }
applyFont(key)
```

Особенности: набор шрифтов закрытый, без свободного ввода названия – иначе можно было бы задать невалидный Google Fonts URL и сломать первую отрисовку страницы; `applyFont` меняет CSS custom properties и подгружает нужный `<link>` на Google Fonts, создавая его при первом вызове и переиспользуя дальше.

frontend/src/theme/seo.js Применяет SEO-настройки сайта (заголовков вкладки, meta-описание, favicon), заданные владельцем, к текущей открытой странице.

```
applySeo(seo)
```

Особенности: поскольку это SPA без серверного рендеринга, `index.html` хранит статичные значения по умолчанию для первой отрисовки и краулеров без JS, а `applySeo` донастраивает DOM уже после загрузки данных из БД.

19 Инфраструктура и деплой

19.1 Два Dockerfile

`backend/Dockerfile` – один этап: устанавливает зависимости из `requirements.txt`, копирует код, создаёт непривилегированного пользователя `appuser` (контейнер не работает от `root`) и при старте сначала накатывает миграции, потом поднимает сервер:

```
RUN mkdir -p /app/uploads && chown appuser:appuser /app/uploads
USER appuser
EXPOSE 8000
CMD ["sh", "-c", "alembic upgrade head && uvicorn app.main:app
↪ --host 0.0.0.0 --port 8000"]
```

Каталог `/app/uploads` создаётся и передаётся во владение `appuser` до первого монтирования именованного тома – новый именованный том Docker наследует права того, что уже лежит по этому пути в образе.

`frontend/Dockerfile` – собран из двух этапов (multi-stage build): сначала `node:26-alpine` собирает статику (`npm run build`), затем результат копируется в чистый `nginx:alpine` – финальный образ не содержит ни Node.js, ни исходников, только готовые файлы и `nginx`:

```
FROM node:26-alpine AS build
...
ARG VITE_SENTRY_DSN
ENV VITE_SENTRY_DSN=$VITE_SENTRY_DSN
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
USER nginx
EXPOSE 8080
```

`VITE_SENTRY_DSN` передаётся как ARG сборки, а не обычная переменная окружения контейнера – значения `import.meta.env.VITE_*` у Vite запекаются прямо в JS-бандл на этапе `npm run build` (раздел 13.1) и не читаются заново при старте контейнера; смена DSN требует пересборки образа, одного `restart` недостаточно.

`frontend/nginx.conf` внутри этого контейнера решает две задачи: отдаёт собранный SPA (`index.html` на любой путь – Vue Router работает в history-режиме) и прозрачно проксирует `/api/*` дальше на контейнер `backend:8000` – благодаря этому браузер всегда обращается к одному и тому же origin, и настоящий CORS между фронтендом и API вообще не участвует в проде.

19.2 `docker-compose.yml`: пять сервисов плюс бэкапы

Продовая сборка всего стека одной командой (`docker compose up - -build -d`) поднимает:

Таблица 12 – Сервисы `docker-compose.yml`

Сервис	Роль
db	<code>postgres:16-alpine.healthcheck – pg_isready</code> – остальные сервисы ждут именно его, не просто запуска контейнера.
backend	Собирается из <code>backend/Dockerfile</code> . Требует обязательных <code>SECRET_KEY/SETUP_TOKEN</code> в переменных окружения – синтаксис <code>\${SECRET_KEY:?Задайте...}</code> в <code>compose</code> -файле сам не даст стеку подняться без них (раздел 3.1).
frontend	Собирается из <code>frontend/Dockerfile</code> , ждёт здорового <code>backend</code> .
caddy	Единственный сервис, торчащий наружу (порты 80/443). Реверс-прокси и автоматический TLS-сертификат Let's Encrypt – раздел 19.3.

Сервис	Роль
<code>backup-local</code>	Ежедневный сжатый <code>pg_dump</code> в <code>./backups</code> (<code>bind-mount</code> , не именованный том – бэкапы видны и забираются снаружи <code>Docker</code>). Включён всегда, донастройки не требует.
<code>backup-s3</code>	Тот же дамп в S3-совместимое хранилище – переживает потерю самого VPS. Выключен по умолчанию (<code>profiles: ["s3-backup"]</code>) – без заполненных <code>S3_BACKUP_*</code> в <code>.env</code> включать не нужно.

Все пять «рабочих» сервисов используют `depends_on: condition: service_healthy`, а не просто порядок запуска – `backend` не стартует, пока `Postgres` не готов принимать соединения; `frontend` – пока `backend` не ответит 200 на `/health` (раздел 3.1); `Caddy` – пока не поднимется `frontend`. `healthcheck` самого `backend` использует `127.0.0.1`, а не `localhost` – в контейнере от непривилегированного пользователя IPv6-листенер не всегда успевает подняться, и `localhost` может резолвиться в `::1` первым, получая `connection refused`, хотя сервис реально слушает на IPv4 (задокументированное наблюдение из самого файла).

19.3 Caddy и nginx: кто кого проксирует

Путь запроса от браузера до `backend` проходит два прокси подряд: браузер → `Caddy` (80/443) → `frontend` (nginx, 8080) → `backend` (uvicorn, 8000). `Caddyfile` в корне репозитория – единственная точка входа снаружи, поэтому и единственное место, где выставляются `security`-заголовки на весь сайт разом:

```
{${SITE_ADDRESS}} {
    header {
        Strict-Transport-Security "max-age=31536000;
        ↪ includeSubDomains"
        X-Frame-Options "DENY"
        Content-Security-Policy "default-src 'self'; script-src
        ↪ 'self' https://mc.yandex.ru; ..."
```

```

        -Server
    }
    reverse_proxy frontend:8080
}

```

`{$SITE_ADDRESS}` – без домена (переменная не задана, дефолт `":80"`) Caddy отдаёт обычный HTTP на 80-м порту – так работает самый первый деплой по голому IP сервера. Как только домен указывает на сервер (А-запись DNS), достаточно задать `SITE_ADDRESS=your-domain.ru` в `.env` и перезапустить Caddy – сертификат Let's Encrypt получается автоматически, без единой правки самого Caddyfile.

19.4 CI: восемь параллельных проверок

`.github/workflows/ci.yml` запускает восемь независимых job'ов на каждый push/PR в `main` – в основном те же команды, что уже встречались в разделе 12, только теперь на настоящем CI-раннере, а не на машине разработчика:

Таблица 13 – Job'ы GitHub Actions

Job	Что делает
backend-tests	<code>pytest -q</code> (юнит-тесты, без БД) + <code>pip-audit</code> на уязвимости зависимостей.
backend-lint	<code>ruff check app tests</code> – только проверка, без автоформатирования (раздел 21 – почему).
backend-integration-tests	Поднимает <code>postgres:16-alpine</code> сервисом самого GitHub Actions, гоняет <code>pytest -m integration tests/integration -q</code> (раздел 12).

Job	Что делает
frontend-build	<code>npm run build</code> (ловит синтаксические ошибки/битые импорты) + <code>npm audit --audit-level=high</code> (не блокирует пайплайн – известные находки в dev-зависимостях сборки, не в проде).
frontend-unit-tests	<code>npm run test:unit</code> (Vitest – хранилища и composables с замкнутым API).
frontend-e2e	Поднимает Postgres, накатывает миграции, запускает настоящий backend, затем <code>npm run playwright test</code> – сквозные сценарии в браузере и регрессия доступности (axe-core).
frontend-lint	<code>npm run lint</code> (ESLint, без prettier --check).
compose-validate	<code>docker compose config --quiet</code> с фиктивными SECRET_KEY/SETUP_TOKEN – проверяет только синтаксис и интерполяцию переменных в <code>docker-compose.yml</code> , не реальный запуск.

Автоматического деплоя по CI нет – ни один из этих job’ов не разворачивает код на сервере, только проверяет его.

19.5 Обновление прод-версии: `scripts/deploy.sh`

Однокомандный скрипт, который забирает свежий код, пересобирает и перезапускает стек, дожидаясь всех healthcheck-ов прежде, чем считать деплой успешным:

```
git pull --ff-only
docker compose up --build -d --wait --wait-timeout 120
docker compose ps
```

Флаг `--wait` – ключевая часть: команда не завершится успешно, пока все

сервисы не станут `healthy`, поэтому `set -euo pipefail` в начале скрипта гарантирует ненулевой код выхода, если что-то не поднялось – сломанный деплой не остаётся тихо висеть незамеченным. Отдельного шага для миграций нет – они накатываются автоматически при старте контейнера `backend` (раздел 19.1). Это осознанно упрощённая замена полноценного CD – нет автозапуска на `merge/push` в `main` и нет автоматического отката; обновление прод-сервера всегда выполняется вручную запуском этого скрипта.

20 Как запустить проект локально

Кратко – то же самое, что подробнее описано в README.md проекта (раздел «Быстрый старт»); здесь – версия для читателя, который уже прочитал весь этот документ и понимает, что запускает.

20.1 Вариант 1: Docker, весь стек одной командой

```
cp .env.example .env # заполните SECRET_KEY/SETUP_TOKEN, см.  
→ комментарии в файле  
docker compose up --build -d
```

Приложение – <http://localhost>, Swagger – <http://localhost/api/docs> (раздел 19.3 объясняет, почему именно через Caddy, а не напрямую на порт backend). Миграции накатываются автоматически. Сразу после первого старта откройте <http://localhost/setup> – это создаёт первого владельца сети (раздел 7, SetupService) и дальше уже недоступно повторно.

20.2 Вариант 2: backend и frontend раздельно, без Docker

Требуется Python 3.14, Node.js 18+, PostgreSQL 13+ (с правами на CREATE EXTENSION – нужно расширение btree_gist для защиты от двойного бронирования, раздел 11.3). Backend:

```
cd backend  
python -m venv venv && source venv/bin/activate  
pip install -r requirements.txt  
cp .env.example .env # заполните под своё окружение  
alembic upgrade head # создаёт схему, расширение btree_gist и  
→ EXCLUDE-ограничение  
python run.py # http://localhost:8000 · Swagger:  
→ /api/docs
```

Frontend (в отдельном терминале):

```
cd frontend  
npm install
```

```
npm run dev    # http://localhost:5173, /api проксируется на :8000
↳ (см. vite.config.js)
```

В этом режиме проксирование `/api` делает уже не `nginx` (раздел 19.1), а сам `dev-сервер Vite` – смотрите блок `server.proxy` в `frontend/vite.config.js`.

20.3 Письма и вход через VK ID на локальной машине

Письма (сброс пароля, подтверждение email, напоминания – раздел 10 и 7) уходят по SMTP; дефолты `config.py` (раздел 3.1) уже нацелены на `Mailpit` – фейковый SMTP-сервер, перехватывающий письма вместо реальной отправки:

```
docker compose -f docker-compose.dev.yml up -d    # поднимает
↳ mailpit
```

Запросите сброс пароля на `/password-reset` – письмо появится на `http://localhost:8025`, никуда реально не уйдёт.

Вход через VK ID (раздел 10.4) требует зарегистрированное приложение на `id.vk.com/business`: redirect URI – `http://localhost:8000/api/auth/vk/callback` (VK ID разрешает `localhost` только на этапе разработки), `client_id` – в переменную `VK_CLIENT_ID` (`backend/.env`). Без неё кнопка «Войти через VK» на фронтенде сама скрывается (`GET /api/auth/vk/enabled`, раздел 15) – функциональность факультативна, приложение прекрасно работает и без неё.

21 Куда смотреть дальше

Этот документ разбирает код. Если нужно что-то другое – вот куда идти:

Таблица 14 – Остальная документация проекта

Документ	Когда открывать
README .md	Быстрый обзор возможностей по ролям, инструкции быстрого запуска (раздел 20 этого документа – их сокращённая версия), описание тестов, деплоя, бэкапов, Sentry, известных ограничений. Первое, что стоит открыть человеку, не читавшему вообще ничего о проекте.
ROADMAP .md	Большой рабочий документ: полный аудит недостатков проекта, план доработки для продакшена по фазам, подробная техническая спецификация функционала «сеть точек» (роль <i>owner</i>). Рабочие заметки по ходу разработки, не продуктовый документ для внешнего читателя – но полезен, если нужно понять, <i>почему</i> что-то устроено именно так (в коде часто встречаются ссылки вида «ROADMAP.md §4.8»).
new_issues.md	Журнал уже разобранных замечаний по проекту – что было найдено и как исправлено. История, а не список открытых задач.
SECURITY .md	Как сообщить об уязвимости (не через публичный GitHub Issue).

Документ	Когда открывать
<code>final_report/main.tex</code>	Итоговый отчёт о разработке для руководителя практики – что было поставлено, как решено, какие архитектурные решения приняты и почему, чем фактически реализованная система отличается от изначально запланированной технического задания. Читать, если нужен обзор <i>продукта</i> , а не кода – этот документ специально его не дублирует.
<code>tech_zadanie/main.tex</code>	Техническое задание – первоначальная постановка задачи, полный перечень функциональных требований (карточки ФТ-001–ФТ-015).
<code>user_manual/main.tex</code>	Руководство пользователя – пошаговые инструкции по работе с системой для всех четырёх ролей.
<code>admin_manual/main.tex</code>	Руководство администратора – развёртывание, эксплуатация, резервное копирование серверной части (подробная версия раздела 19–20 этого документа).
<code>notes/</code>	Рабочие заметки по ходу разработки (анализ технологического стека, черновик спецификации требований, подсказки для дальнейшей разработки). Не входит в состав официальной документации проекта.

Документ	Когда открывать
graphify-out/GRAPH_REPORT.md	Автоматически сгенерированный граф связей кода (инструмент <code>graphify</code>) – какие классы и функции сильнее всего связаны друг с другом, какие сообщества файлов формируют логические модули. Полезен как альтернативная, менее структурированная карта того же кода, что описан в этом документе.